

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



BÁO CÁO ĐỒ ÁN MÔN HỌC

HỌC PHẦN: LẬP TRÌNH THIẾT BỊ DI ĐỘNG

**ĐỀ TÀI: ỨNG DỤNG TÌM CHỖ, HỌC TẬP VÀ KẾT NỐI
SFINITY**

Nhóm sinh viên thực hiện:

1. Phạm Lê Anh Khoa – 6451071035 – CQ.64.CNTT
2. Ngô Thành Khôi – 6451071038 – CQ.64.CNTT
3. Lê Gia Luân – 6451071045 – CQ.64.CNTT
4. Đinh Công Vinh Quang – 6451071061 – CQ.64.CNTT

Giảng viên hướng dẫn: Th.S Nguyễn Thiện Dương

Tp. Hồ Chí Minh, năm 2026

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHÂN HIỆU TẠI TP. HỒ CHÍ MINH
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



**BÁO CÁO ĐỒ ÁN MÔN HỌC
HỌC PHẦN: LẬP TRÌNH THIẾT BỊ DI ĐỘNG
ĐỀ TÀI: ỨNG DỤNG TÌM CHỖ HỌC TẬP VÀ KẾT NỐI
SFINITY**

Nhóm sinh viên thực hiện:

1. Phạm Lê Anh Khoa – 6451071035 – CQ.64.CNTT
2. Ngô Thành Khôi – 6451071038 – CQ.64.CNTT
3. Lê Gia Luân – 6451071045 – CQ.64.CNTT
4. Đinh Công Vinh Quang – 6451071061 – CQ.64.CNTT

Giảng viên hướng dẫn: Th.S Nguyễn Thiện Dương

Tp. Hồ Chí Minh, năm 2026

LỜI CẢM ƠN

Lời đầu tiên, chúng em xin bày tỏ lòng tri ân sâu sắc đến quý thầy cô Bộ môn Công nghệ Thông Tin – Phân hiệu Trường Đại học Giao thông Vận tải tại Thành phố Hồ Chí Minh. Nhờ sự tận tâm chỉ dạy và những kiến thức quý báu mà thầy cô đã truyền đạt trong suốt thời gian học tập tại trường, chúng em mới có đủ nền tảng vững chắc để hoàn thành đồ án này.

Đặc biệt, chúng em xin gửi lời cảm ơn chân thành đến thầy Nguyễn Thiện Dương, người đã trực tiếp hướng dẫn và đồng hành cùng chúng em trong suốt quá trình thực hiện đồ án. Thầy không chỉ giúp chúng em định hướng đề tài, cung cấp tài liệu tham khảo mà còn tận tình giải đáp những thắc mắc, tháo gỡ khó khăn và truyền cảm hứng để chúng em hoàn thành tốt nhiệm vụ của mình. Sự chỉ bảo tận tâm và sự động viên của thầy chính là động lực to lớn để chúng em vượt qua những thử thách.

Bên cạnh đó, chúng em cũng muốn gửi lời cảm ơn đến bạn bè và đồng đội – những người đã luôn sát cánh, chia sẻ và hỗ trợ lẫn nhau trong suốt quá trình học tập và thực hiện đồ án này.

Mặc dù đã cố gắng hết sức để hoàn thành bài tập với tinh thần trách nhiệm cao, nhưng do giới hạn về thời gian và kinh nghiệm, bài làm chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự cảm thông và những ý kiến đóng góp quý báu từ quý thầy cô để có thể hoàn thiện bài làm tốt hơn trong tương lai.

Cuối cùng, chúng em xin kính chúc thầy Nguyễn Thiện Dương nói riêng và quý thầy cô Bộ môn Công nghệ Thông Tin nói chung luôn dồi dào sức khỏe, hạnh phúc và gặt hái nhiều thành công trong sự nghiệp giảng dạy.

Chúng em xin chân thành cảm ơn!

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Tp. Hồ Chí Minh, ngày ... tháng ... năm 2026

Giảng viên hướng dẫn

ThS. Nguyễn Thiện Dương

MỤC LỤC

LỜI CẢM ƠN	ii
NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN.....	iii
MỤC LỤC	iv
DANH MỤC CHỮ VIẾT TẮT.....	viii
DANH MỤC HÌNH ẢNH	ix
BẢNG PHÂN CÔNG CÔNG VIỆC	xi
CHƯƠNG 1 TỔNG QUAN	1
1.1 Giới thiệu đề tài	1
1.2 Mục tiêu đề tài	1
1.2.1 Mục tiêu lý thuyết	1
1.2.2 Mục tiêu về thực nghiệm	2
1.3 Mục tiêu nghiên cứu	2
1.4 Đối tượng nghiên cứu	2
1.5 Phạm vi nghiên cứu	3
1.6 Phương pháp nghiên cứu	4
1.7 Ý nghĩa thực tiễn của đề tài	4
1.8 Cấu trúc cuốn báo cáo đồ án.....	5
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT	6
2.1 Khái niệm về nghiệp vụ trong lĩnh vực đề tài	6
2.1.1 Ứng dụng chia sẻ tài nguyên học tập trong cộng đồng sinh viên	6
2.1.2 Kiến trúc ứng dụng di động kết hợp web admin.....	6
2.2 Các công trình liên quan	6
2.2.1 Ứng dụng StudySpot (2022)	7

2.2.2 Nền tảng Studocu (2013 – nay)	7
2.2.3 Nhận định chung và đóng góp của tác giả	7
2.3 Các công nghệ áp dụng trong đề tài.....	7
2.3.1 Flutter	7
2.3.2 React (TypeScript)	9
2.3.3 Node.js và Express	9
2.3.4 Firebase	10
2.4 Kết luận phạm vi bài toán và công nghệ áp dụng.....	11
CHƯƠNG 3 PHÂN TÍCH THIẾT KẾ HỆ THỐNG	13
3.1 Mô tả bài toán	13
3.1.1 Bối cảnh thực tế	13
3.1.2 Khó khăn và hạn chế của phương pháp hiện tại	13
3.1.3 Nhu cầu xây dựng ứng dụng	13
3.1.4 Mục tiêu hệ thống	14
3.1.5 Đối tượng sử dụng và phạm vi.....	14
3.2 Phân tích yêu cầu hệ thống	14
3.2.1 Yêu cầu chức năng.....	14
3.2.2 Yêu cầu phi chức năng	15
3.3 Thiết kế giao diện ứng dụng	15
3.4 Phân tích và thiết kế hệ thống.....	21
3.4.1 Sơ đồ Use Cases.....	22
3.4.2 Biểu đồ hoạt động (Activity Diagram)	23
3.4.3 Biểu đồ tuần tự (Sequence Diagram).....	30
3.4.4 Biểu đồ lớp	35
3.5 Thiết kế cơ sở dữ liệu	37

3.6 Thiết kế kiến trúc hệ thống	39
CHƯƠNG 4 CÀI ĐẶT THỰC NGHIỆM VÀ ĐÁNH GIÁ.....	41
4.1 Cài đặt hệ thống	41
4.1.1 Môi trường và công nghệ cài đặt	41
4.1.2 Cấu trúc project và tổ chức source code	41
4.1.3 Kiến trúc cài đặt và mô hình hoạt động của hệ thống.....	43
4.1.4 Cài đặt ứng dụng mobile	44
4.1.5 Cài đặt backend và các module API	45
4.1.6 Cài đặt cơ chế xử lý dữ liệu	46
4.1.7 Các module chức năng chính đã thực hiện	47
4.1.8 Gợi ý minh họa hình ảnh giao diện trong báo cáo	49
4.2 Mô tả quá trình sử dụng công cụ AI hỗ trợ phát triển hệ thống	52
4.2.1 Lý do lựa chọn công cụ AI.....	53
4.2.2 AI hỗ trợ trong từng giai đoạn phát triển	53
4.2.3 Cách nhóm sử dụng prompt và kiểm soát chất lượng.....	54
4.2.4 Vai trò thực tế của người phát triển	56
4.2.5 Đánh giá hiệu quả của AI trong dự án	56
4.3 Thực nghiệm hệ thống	57
4.3.1 Mục tiêu thực nghiệm	57
4.3.2 Môi trường thực nghiệm	57
4.3.3 Các kịch bản thực nghiệm chức năng	58
4.3.4 Kiểm thử giao diện và trải nghiệm sử dụng.....	59
4.3.5 Kiểm thử hiệu năng và ổn định.....	60
4.4 Đánh giá hệ thống	60
4.4.1 Mức độ hoàn thiện chức năng.....	60

4.4.2 Đánh giá về kiến trúc và tổ chức hệ thống.....	61
4.4.3 Đánh giá về hiệu năng và tính ổn định	61
4.4.4 Đánh giá về trải nghiệm người dùng	61
4.4.5 Ưu điểm của hệ thống	62
4.4.6 Hạn chế của hệ thống	62
4.4.7 Nguyên nhân của hạn chế	62
4.5 Khó khăn và hạn chế trong quá trình triển khai	63
CHƯƠNG 5 KẾT LUẬN.....	64
5.1 Kết quả đạt được	64
5.1.1 Về mặt lý thuyết.....	64
5.1.2 Về mặt thực nghiệm	64
5.2 Hạn chế còn tồn đọng	65
5.3 Hướng phát triển trong tương lai	66
TÀI LIỆU THAM KHẢO.....	68

DANH MỤC CHỮ VIẾT TẮT

STT	Mô tả	Ý nghĩa	Ghi chú
1	CSDL	Cơ Sở Dữ Liệu	
2	SQL	Structured Query Language	
3	API	Application Programming Interface	
4	JWT	JSON Web Token	
5	UI	User Interface	
6	RBAC	Role-Based Access Control	
7	UML	Unified Modeling Language	

DANH MỤC HÌNH ẢNH

Hình 2.1 Logo Flutter	9
Hình 2.2 Logo React.....	9
Hình 2.3 Logo Node Js	10
Hình 2.4 Logo Firebase	11
Hình 3.1 Giao diện đăng nhập/ đăng ký	16
Hình 3.2 Giao diện màn hình Trang chủ	17
Hình 3.3 Giao diện bản đồ địa điểm	18
Hình 3.4 Giao diện tài liệu.....	19
Hình 3.5 Giao diện cộng đồng.....	20
Hình 3.6 Giao diện hồ sơ và cài đặt.....	21
Hình 3.7 Biểu đồ Use Cases cho vai trò quản trị viên.....	22
Hình 3.8 Biểu đồ Use Cases cho người dung	23
Hình 3.9 Biểu đồ hoạt động của đăng nhập.....	24
Hình 3.10 Biểu đồ hoạt động của đăng ký.....	25
Hình 3.11 Biểu đồ hoạt động checking địa điểm học tập.....	26
Hình 3.12 Biểu đồ hoạt động điểm học gần tôi	27
Hình 3.13 Biểu đồ hoạt động tạo tài liệu	28
Hình 3.14 Biểu đồ hoạt động duyệt tài liệu	29
Hình 3.15 Biểu đồ hoạt động đánh giá địa điểm	30
Hình 3.16 Biểu đồ tuần tự đăng nhập	31
Hình 3.17 Biểu đồ tuần tự đăng ký.....	32
Hình 3.18 Biểu đồ tuần tự quên mật khẩu	33
Hình 3.19 Biểu đồ tuần tự checkin và đánh giá.....	34

Hình 3.20 Biểu đồ tuần tự tạo tài liệu	35
Hình 3.21 Biểu đồ lớp cho tính năng xác thực	36
Hình 3.22 Biểu đồ lớp cho tính năng quản lý	37
Hình 4.1 Giao diện đăng nhập / đăng ký trên ứng dụng thực tế	49
Hình 4.2 Giao diện trang chủ và khám phá trên ứng dụng thực tế	50
Hình 4.3 Giao diện tìm kiếm địa điểm trên ứng dụng thực tế	50
Hình 4.4 Giao diện chi tiết địa điểm trên ứng dụng thực tế	50
Hình 4.5 Giao diện danh sách tài liệu trên ứng dụng thực tế	51
Hình 4.6 Giao diện nhóm học tập trên ứng dụng thực tế	51
Hình 4.7 Giao diện chat nhóm trên ứng dụng thực tế	52
Hình 4.8 Giao diện hồ sơ cá nhân và cài đặt	52
Hình 4.9 Ví dụ yêu cầu sử dụng AI	55

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	Họ và Tên	Nội dung công việc	Chữ ký	Phần trăm đóng góp (%)
1	Ngô Thành Khôi	Trang chủ, Địa điểm, Check-in học tập		30%
2	Phạm Lê Anh Khoa	Chức năng chia sẻ tài liệu Chức năng nhóm học tập Chức năng bạn bè		30%
3	Lê Gia Luân	Chức năng đổi ngôn ngữ, giao diện, thông báo, quản lý hồ sơ Web quản trị viên: quản lý tài liệu, địa điểm Làm báo cáo		20%
4	Đinh Công Vinh Quang	Làm Onboarding, chức năng phản hồi Web quản trị viên: Phản hồi, báo cáo, admin, dashboard, cài đặt, mã OTP đổi password Làm báo cáo, sửa lỗi nhỏ		20%

CHƯƠNG 1 TỔNG QUAN

1.1 Giới thiệu đề tài

Trong bối cảnh chuyển đổi số mạnh mẽ hiện nay, nhu cầu tiếp cận tài liệu học tập và không gian học tập ngày càng tăng, đặc biệt trong cộng đồng sinh viên và người học trực tuyến. Các nền tảng hiện tại thường phân tán giữa nhiều ứng dụng riêng lẻ: quản lý tài liệu trên một nền tảng, tìm kiếm địa điểm học tập trên một ứng dụng khác, và kết nối cộng đồng học tập trên một nền tảng thứ ba. Điều này tạo ra rào cản về trải nghiệm người dùng, thiếu tính đồng bộ và khó quản lý nội dung tập trung.

Xu hướng ứng dụng di động cross-platform kết hợp hệ thống quản trị web và API phía server đang được nhiều dự án lựa chọn nhờ khả năng chia sẻ logic nghiệp vụ, đồng nhất trải nghiệm người dùng, và dễ dàng mở rộng. Đồng thời, các công nghệ như Firebase Authentication, Prisma ORM, và Mapbox/OpenStreetMap mang lại khả năng xác thực an toàn, quản lý dữ liệu hiệu quả, và dịch vụ bản đồ không phụ thuộc nền tảng.

Từ thực tiễn đó, đề tài Sfinity được phát triển nhằm xây dựng một hệ thống tích hợp toàn diện, đồng bộ, phục vụ nhu cầu học tập và kết nối cộng đồng trong một giải pháp duy nhất.

1.2 Mục tiêu đề tài

1.2.1 Mục tiêu lý thuyết

Để thực hiện đề tài, việc tìm hiểu và nắm vững kiến thức về mô hình phát triển ứng dụng cross-platform là yêu cầu bắt buộc, cụ thể là framework Flutter của Google cho phép xây dựng một codebase duy nhất chạy trên cả Android và iOS. Bên cạnh đó, việc nghiên cứu React kết hợp Vite làm công cụ build cho phần web admin dashboard cũng cần thiết, giúp tạo ra giao diện quản trị nhanh chóng và chuyên nghiệp. Trên phía backend, đề tài yêu cầu nghiên cứu sâu về Express.js để xây dựng RESTful API. Mô hình xác thực JWT (JSON Web Token) và cơ chế phân quyền RBAC (Role-Based Access Control) cũng được nghiên cứu kỹ lưỡng nhằm đảm bảo hệ thống an toàn và phân biệt rõ ràng quyền hạn giữa người dùng thường và quản trị viên.

Ngoài ra, đề tài cũng cần tìm hiểu cách tích hợp Firebase Authentication hỗ trợ đăng nhập bằng email/password và Google Sign-In trên ứng dụng di động, cũng như

tích hợp thư viện `flutter_map` dựa trên `OpenStreetMap` để hiển thị bản đồ tương tác phục vụ tính năng địa điểm và tìm kiếm không gian học tập. Cuối cùng, mô hình tổ chức monorepo - nơi cả ba thành phần mobile, web-admin và backend cùng tồn tại trong một repository.

1.2.2 Mục tiêu về thực nghiệm

Về mặt triển khai, đề tài hướng đến việc xây dựng hoàn chỉnh ứng dụng di động Sfinity với đầy đủ các chức năng phục vụ người dùng cuối: đăng ký và đăng nhập, xem và quản lý tài liệu học tập, khám phá địa điểm học tập trên bản đồ tương tác, tìm kiếm không gian học tập gần vị trí hiện tại, lưu nội dung yêu thích, xem lịch sử hoạt động, kết bạn và quản lý danh sách bạn bè, tạo và tham gia nhóm học tập, nhận thông báo trong ứng dụng, gửi phản hồi và báo cáo vi phạm, cùng các cài đặt tài khoản và ứng dụng. Đồng thời, đề tài cũng cần xây dựng trang web cung cấp cho quản trị viên bảng điều khiển thống kê trực quan, quản lý người dùng, quản lý nội dung, quản lý danh mục và tiện ích, xử lý phản hồi và báo cáo vi phạm từ người dùng, gửi thông báo hệ thống, và cấu hình ứng dụng.

Trên phía server, đề tài triển khai Backend REST API hoàn chỉnh bao gồm các endpoint cho tất cả các chức năng trên, middleware xác thực JWT và phân quyền RBAC, module upload file, tài liệu API tự động qua Swagger/OpenAPI, và tích hợp Firebase để gửi thông báo đẩy. Toàn bộ hệ thống được kiểm thử thủ công trên thiết bị Android thực và trình duyệt web nhằm đảm bảo các chức năng hoạt động đúng như thiết kế.

1.3 Mục tiêu nghiên cứu

Đề tài được thực hiện nhằm mục đích xây dựng một hệ thống tích hợp toàn diện cho việc quản lý và chia sẻ tài liệu học tập, kết nối người dùng thông qua các tính năng mạng xã hội học tập và hỗ trợ khám phá không gian học tập thực tế.

1.4 Đối tượng nghiên cứu

Đề tài tập trung nghiên cứu và phát triển các thành phần sau:

- Ứng dụng di động Flutter cho người dùng cuối (Android & iOS), bao gồm các module: xác thực, tài liệu, địa điểm, tìm kiếm không gian học tập, hồ sơ cá nhân, yêu thích, lịch sử, kết bạn, nhóm học tập, thông báo, và cài đặt.
- Web Admin Dashboard dựa trên React + Ant Design, phục vụ quản trị viên quản lý người dùng, nội dung, danh mục, phản hồi, báo cáo, thông báo hệ thống và cấu hình ứng dụng.
- Backend REST API với Express.js, quản lý toàn bộ nghiệp vụ phía server bao gồm xác thực JWT, phân quyền RBAC, CRUD dữ liệu (User, Content, Category, Favorite, Feedback, Friendship, Group, GroupMember, Notification, Report), upload file, và dịch vụ bản đồ.
- Cơ sở dữ liệu SQLite với Firebase quản lý các bảng dữ liệu quan hệ: User, Content, Category, Favorite, Feedback, Friendship, Group, GroupMember, Notification, Report, PasswordReset.

1.5 Phạm vi nghiên cứu

Phạm vi thực hiện:

- Ứng dụng di động hỗ trợ Android và iOS thông qua Flutter (cross-platform).
- Web Admin Dashboard chạy trên nền tảng web (trình duyệt).
- Backend REST API triển khai trên Node.js với cơ sở dữ liệu SQLite.
- Các chức năng cốt lõi: xác thực người dùng (email/password, Google), quản lý tài liệu (CRUD, phân loại, yêu thích), địa điểm trên bản đồ (CRUD, đánh giá, tiện ích kèm theo), tìm kiếm không gian học tập theo vị trí, kết bạn và nhóm học tập, thông báo, phản hồi và báo cáo vi phạm.
- Hệ thống quản trị với đầy đủ chức năng quản lý nội dung, người dùng, và thống kê.

Giới hạn ngoài phạm vi:

- Không triển khai tối ưu hóa hiệu năng ở quy mô lớn (sharding, caching phân tán).

- Không phát triển tính năng thanh toán trực tuyến hoặc mua sắm trong ứng dụng.
- Không tích hợp chat thời gian thực (WebSocket) cho nhóm học tập (hiện chỉ có quản lý nhóm).
- Không triển khai ứng dụng lên các store (Google Play, App Store) và không triển khai hosting thực tế (production deployment).

1.6 Phương pháp nghiên cứu

Trong giai đoạn phân tích và thiết kế, đề tài tìm hiểu tài liệu liên quan và phân tích công nghệ, từ đó xây dựng kiến trúc tổng thể theo mô hình monorepo gồm ba phần: ứng dụng di động, web admin và backend API.

Về backend API, đề tài sử dụng Node.js kết hợp Express.js để xây dựng RESTful API, Prisma ORM để quản lý cơ sở dữ liệu, triển khai JWT và RBAC để xác thực và phân quyền, đồng thời dùng Swagger/OpenAPI để tự động tạo tài liệu API.

Về ứng dụng di động, đề tài sử dụng Flutter để phát triển cross-platform, quản lý trạng thái bằng Provider, tích hợp Firebase Auth, hiển thị bản đồ tương tác, điều hướng ứng dụng và kết nối backend qua HTTP client.

Về web admin, đề tài sử dụng React 19 với Vite, hiển thị biểu đồ thống kê và kết nối API qua axios.

Trong giai đoạn kiểm thử, đề tài sử dụng phương pháp kiểm thử thủ công trên thiết bị Android và trình duyệt web, kiểm tra API qua Swagger.

1.7 Ý nghĩa thực tiễn của đề tài

Về mặt ứng dụng thực tế, hệ thống Sfinity mang lại giá trị đa chiều cho các đối tượng khác nhau. Đối với người dùng (sinh viên, học sinh, người học trực tuyến), ứng dụng di động cung cấp một nền tảng tập trung để truy cập tài liệu học tập, khám phá địa điểm học tập thực tế gần họ, kết nối với bạn bè và tham gia nhóm học tập, giúp tiết kiệm thời gian tìm kiếm và nâng cao hiệu quả học tập. Đối với quản trị viên nội dung, web cung cấp công cụ trực quan để quản lý người dùng, duyệt và quản lý nội dung tài liệu,

địa điểm, xử lý phản hồi và báo cáo vi phạm, nâng cao hiệu quả quản lý so với các phương pháp thủ công truyền thống.

1.8 Cấu trúc cuốn báo cáo đồ án

Báo cáo đồ án được tổ chức thành các chương với bố cục logic như sau:

- Chương 1 — Giới thiệu: Trình bày tổng quan đề tài, bao gồm bối cảnh thực tiễn, mục tiêu, mục đích, đối tượng, phạm vi, phương pháp nghiên cứu, ý nghĩa thực tiễn và cấu trúc báo cáo.
- Chương 2 — Cơ sở lý thuyết: Trình bày các kiến thức nền tảng về Flutter, React, Express.js, Firebase Authentication, JWT, RBAC, và các công nghệ liên quan được sử dụng trong đề tài.
- Chương 3 — Phân tích và thiết kế hệ thống: Trình bày khảo sát thực tế, phân tích yêu cầu, thiết kế kiến trúc tổng thể, và thiết kế cơ sở dữ liệu.
- Chương 4 — Triển khai và thực nghiệm: Trình bày chi tiết quá trình triển khai từng thành phần, các giao diện chính, kết quả thực nghiệm và hình ảnh minh họa.

Kết luận chương: Chương 1 đã trình bày tổng quan về đề tài Sfinity — một hệ thống tích hợp đa nền tảng bao gồm ứng dụng di động Flutter, Web Admin Dashboard React, và Backend REST API Express.js. Đề tài được thực hiện nhằm giải quyết bài toán thực tiễn về việc phân tán nền tảng quản lý tài liệu và kết nối cộng đồng học tập hiện nay. Các mục tiêu về lý thuyết và thực nghiệm đã được xác định rõ ràng, phạm vi nghiên cứu và giới hạn được phân định cụ thể. Phương pháp nghiên cứu kết hợp giữa phân tích thiết kế hệ thống, phát triển phần mềm, và kiểm thử thực nghiệm. Chương này làm tiền đề để chuyển sang các chương tiếp theo: cơ sở lý thuyết về các công nghệ sử dụng, phân tích thiết kế chi tiết hệ thống, và triển khai thực nghiệm.

CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

Tóm tắt chương: Trình bày các khái niệm cơ bản về ứng dụng di động đa nền tảng và dịch vụ dựa trên vị trí, làm nền tảng lý thuyết cho đề tài. Đồng thời, chương khảo sát một số đề tài nghiên cứu và sản phẩm thực tế liên quan, từ đó chỉ ra những hạn chế còn tồn đọng và đề xuất hướng phát triển cho đề tài.

2.1 Khái niệm về nghiệp vụ trong lĩnh vực đề tài

2.1.1 Ứng dụng chia sẻ tài nguyên học tập trong cộng đồng sinh viên

Trong bối cảnh giáo dục hiện đại, nhu cầu trao đổi tài liệu học tập và tìm kiếm không gian học tập phù hợp ngày càng trở nên cấp thiết. Các hệ thống hỗ trợ học tập cộng đồng (Community Learning Support Systems) là những nền tảng cho phép người dùng - chủ yếu là sinh viên - đăng tải, tìm kiếm và đánh giá các nguồn tài nguyên học tập, bao gồm tài liệu, địa điểm học nhóm, không gian yên tĩnh, và các tiện ích hỗ trợ.

Nghiệp vụ cốt lõi của hệ thống Sfinity bao gồm:

- Quản lý địa điểm học tập: Sinh viên có thể đăng ký, đánh giá và lọc các địa điểm học tập dựa trên khu vực, tiện ích (WiFi, ổ cắm, điều hòa...) và trạng thái hoạt động.
- Chia sẻ tài liệu học tập: Người dùng có thể tải lên và tìm kiếm tài liệu theo danh mục (bài giảng, đề thi, ghi chú...).
- Tương tác cộng đồng: Hệ thống hỗ trợ nhóm học tập, bình luận, lưu yêu thích và thông báo real-time.
- Quản trị nội dung: Ban quản trị có thể duyệt, ẩn hoặc xóa nội dung vi phạm, đồng thời theo dõi thống kê hoạt động toàn hệ thống.

2.1.2 Kiến trúc ứng dụng di động kết hợp web admin

Hệ thống được thiết kế theo mô hình tách biệt giữa ứng dụng di động dành cho người dùng cuối và bảng điều khiển web dành cho quản trị viên. Hai thành phần này giao tiếp thông qua một backend API chung, đảm bảo tính nhất quán dữ liệu và khả năng mở rộng trong tương lai.

2.2 Các công trình liên quan

2.2.1 Ứng dụng StudySpot (2022)

StudySpot là ứng dụng di động cho phép sinh viên tại các trường đại học ở Mỹ tìm kiếm và đánh giá không gian học tập trong khuôn viên trường. Ứng dụng tích hợp bản đồ, bộ lọc theo tiện ích và hệ thống đánh giá sao.

Đánh giá: Ứng dụng giải quyết tốt bài toán tìm kiếm địa điểm nhưng còn hạn chế ở chỗ không hỗ trợ chia sẻ tài liệu học tập, không có tính năng nhóm cộng đồng và thiếu hệ thống quản trị nội dung cho ban quản lý nhà trường.

2.2.2 Nền tảng Studocu (2013 – nay)

Studocu là nền tảng web toàn cầu cho phép sinh viên chia sẻ và tải xuống tài liệu học tập theo trường và môn học, hiện có hơn 15 triệu tài liệu được chia sẻ trên toàn thế giới.

Đánh giá: Studocu mạnh về kho tài liệu nhưng không có tính năng địa điểm học tập, không hỗ trợ ứng dụng di động native đủ mạnh và không có cơ chế quản lý cộng đồng địa phương theo từng trường đại học cụ thể.

2.2.3 Nhận định chung và đóng góp của tác giả

Qua khảo sát các sản phẩm trên, có thể nhận thấy chưa có hệ thống nào kết hợp đồng thời cả hai chức năng: chia sẻ tài liệu học tập và khám phá địa điểm học tập trong một ứng dụng di động dành riêng cho sinh viên Việt Nam, kèm theo một bảng điều khiển quản trị web đầy đủ tính năng.

Đóng góp chính của đề tài này bao gồm:

- Xây dựng hệ thống tích hợp cả địa điểm và tài liệu học tập trong một nền tảng thống nhất.
- Phát triển tính năng nhóm học tập với cơ chế phê duyệt thành viên.
- Xây dựng bảng quản trị web với đầy đủ chức năng kiểm duyệt nội dung, quản lý người dùng và thống kê hoạt động.
- Tích hợp hệ thống thông báo real-time và đồng bộ dữ liệu đa thiết bị.

2.3 Các công nghệ áp dụng trong đề tài

2.3.1 Flutter

Flutter là framework mã nguồn mở của Google dùng để xây dựng ứng dụng di động đa nền tảng (iOS và Android) từ một codebase duy nhất bằng ngôn ngữ Dart [1]. Các thành phần chính của flutter bao gồm:

- Flutter SDK: là bộ công cụ phát triển phần mềm cung cấp các thư viện và công cụ cần thiết để xây dựng các ứng dụng Flutter, gồm có:
 - Dart SDK: là ngôn ngữ lập trình được sử dụng để viết các ứng dụng Flutter. Dart SDK cung cấp trình biên dịch và các công cụ cần thiết để làm việc với Dart.
 - Flutter Engine: là lõi của Flutter, được viết bằng C++, cung cấp các công cụ cần thiết để render giao diện người dùng và thực hiện các tác vụ như xử lý sự kiện, vẽ đồ họa và quản lý các thành phần UI.
 - Framework: Framework của Flutter được viết bằng Dart, bao gồm các thư viện và widget để xây dựng giao diện người dùng, quản lý trạng thái, điều hướng và nhiều chức năng khác.
- Widgets: là thành phần cơ bản của giao diện người dùng trong Flutter. Mọi thứ trong Flutter đều là widget, từ các thành phần UI cơ bản như nút bấm và văn bản đến các cấu trúc phức tạp như bố cục và hoạt ảnh. Flutter cung cấp hai loại widget chính:
 - Stateless Widgets: Widgets không thay đổi trạng thái trong suốt vòng đời của chúng. Chúng chỉ phụ thuộc vào dữ liệu đầu vào và hiển thị giao diện người dùng dựa trên đó.
 - Stateful Widgets: Widgets có thể thay đổi trạng thái trong suốt vòng đời của chúng. Chúng có thể phản ứng với các sự kiện và cập nhật giao diện người dùng dựa trên trạng thái hiện tại.

Flutter sử dụng cơ chế render riêng thông qua Skia/Impeller engine, giúp giao diện nhất quán trên mọi nền tảng và đạt hiệu năng 60fps. Trong đề tài này, Flutter được dùng để xây dựng toàn bộ ứng dụng di động Sfinity.



Hình 2.1 Logo Flutter

2.3.2 React (TypeScript)

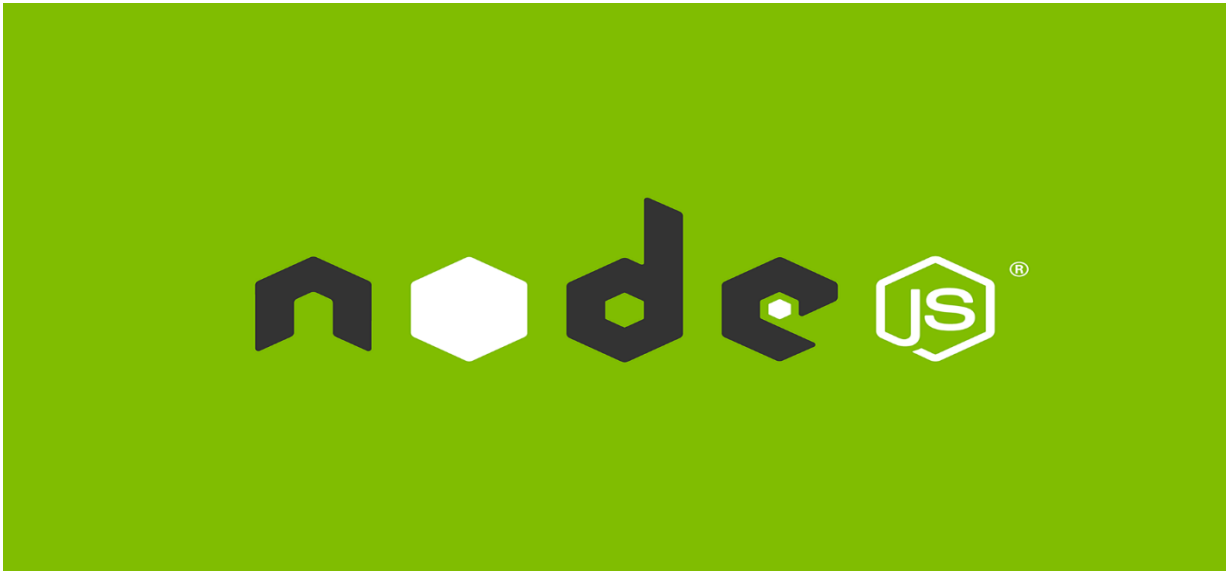
React là thư viện JavaScript mã nguồn mở do Meta phát triển, sử dụng kiến trúc component-based và Virtual DOM để xây dựng giao diện người dùng web hiệu quả [2]. Kết hợp với TypeScript, React cung cấp khả năng kiểm tra kiểu tĩnh giúp giảm lỗi runtime. Trong đề tài, React + TypeScript được dùng để xây dựng bảng điều khiển quản trị web-admin.



Hình 2.2 Logo React

2.3.3 Node.js và Express

Node.js là môi trường runtime JavaScript phía máy chủ, cho phép xây dựng các ứng dụng backend hiệu năng cao với mô hình non-blocking I/O [3]. Express là framework tối giản chạy trên Node.js, hỗ trợ định tuyến, middleware và xây dựng RESTful API nhanh chóng. Backend của Sfinity được xây dựng trên nền tảng này, cung cấp API cho cả ứng dụng di động và web admin.



Hình 2.3 Logo Node Js

2.3.4 Firebase

Firebase là nền tảng Backend-as-a-Service (BaaS) của Google, cung cấp các dịch vụ như Authentication, Firestore (NoSQL database), Cloud Storage và Cloud Messaging [4]. Trong đề tài, Firebase được sử dụng cho xác thực người dùng (Google, email), lưu trữ dữ liệu người dùng và gửi thông báo đẩy (push notification).



2.4 Kết luận phạm vi bài toán và công nghệ áp dụng

Qua quá trình khảo sát các hệ thống hỗ trợ học tập hiện có như StudySpot và Studocu, có thể nhận thấy rằng mỗi nền tảng chỉ tập trung giải quyết một phần nhu cầu của sinh viên. Một số hệ thống hỗ trợ tìm kiếm địa điểm học tập nhưng không cung cấp chức năng chia sẻ tài liệu; ngược lại, các nền tảng chia sẻ tài liệu lại chưa hỗ trợ quản lý và đánh giá không gian học tập cũng như tương tác cộng đồng theo phạm vi từng trường đại học. Bên cạnh đó, các hệ thống hiện nay thường thiếu cơ chế quản trị tập trung, kiểm duyệt nội dung và hỗ trợ hoạt động cộng đồng một cách toàn diện.

Từ thực tế đó, đề tài hướng đến giải quyết bài toán xây dựng một hệ thống hỗ trợ học tập cộng đồng dành cho sinh viên, cho phép tích hợp đồng thời việc chia sẻ tài liệu học tập, tìm kiếm và đánh giá địa điểm học tập, tổ chức nhóm học tập và tương tác giữa các thành viên trong cùng một nền tảng thống nhất. Hệ thống được triển khai dưới dạng ứng dụng di động kết hợp với bảng điều khiển quản trị trên nền web nhằm đáp ứng nhu cầu của cả người dùng cuối và nhà quản lý.

Đóng góp chính của đề tài bao gồm:

- Tích hợp hai chức năng quan trọng là chia sẻ tài liệu học tập và quản lý địa điểm học tập trong cùng một hệ thống.
- Xây dựng cơ chế cộng đồng học tập với các tính năng nhóm học tập, bình luận, lưu yêu thích và thông báo thời gian thực.
- Phát triển bảng điều khiển quản trị web hỗ trợ quản lý người dùng, kiểm duyệt nội dung và thống kê hoạt động hệ thống.
- Tối ưu hóa khả năng đồng bộ dữ liệu đa thiết bị thông qua nền tảng điện toán đám mây.
- Ứng dụng kiến trúc Client-Server hiện đại giúp hệ thống dễ dàng mở rộng và bảo trì trong tương lai.

Để giải quyết bài toán đặt ra, đề tài lựa chọn các công nghệ và nền tảng sau:

- Flutter và ngôn ngữ Dart để phát triển ứng dụng di động đa nền tảng Android và iOS.
- React kết hợp TypeScript để xây dựng bảng điều khiển quản trị web.
- Node.js và Express để xây dựng hệ thống backend theo kiến trúc RESTful API.
- Firebase Authentication để quản lý xác thực người dùng.
- Firebase Firestore làm cơ sở dữ liệu NoSQL lưu trữ dữ liệu hệ thống.
- Firebase Storage lưu trữ tài liệu học tập và hình ảnh.
- Firebase Cloud Messaging (FCM) cung cấp chức năng thông báo đẩy thời gian thực.
- JWT (JSON Web Token) được sử dụng để xác thực và phân quyền truy cập API.

Như vậy, đề tài không chỉ giải quyết nhu cầu chia sẻ tài nguyên học tập của sinh viên mà còn tạo ra một môi trường học tập cộng đồng hiện đại, hỗ trợ tìm kiếm không gian học tập phù hợp, tăng cường tương tác giữa người học và nâng cao hiệu quả quản lý nội dung trên nền tảng số.

Kết luận chương: Chương 2 đã trình bày đầy đủ nền tảng lý thuyết cần thiết cho việc triển khai hệ thống Sfinity, bao gồm các khái niệm nghiệp vụ cốt lõi, phân tích các sản phẩm tương tự đã có trên thị trường và tổng quan về các công nghệ được lựa chọn. Qua đó, tác giả xác định rõ khoảng trống mà đề tài hướng đến lấp đầy, đồng thời làm rõ phạm vi bài toán, đầu vào, đầu ra và stack công nghệ sẽ sử dụng xuyên suốt quá trình phát triển.

CHƯƠNG 3 PHÂN TÍCH THIẾT KẾ HỆ THỐNG

Tóm tắt chương: Chương này trình bày toàn bộ quá trình phân tích và thiết kế hệ thống Sfinity, từ việc mô tả bài toán thực tế về nhu cầu tìm địa điểm học tập và chia sẻ tài liệu của sinh viên, xác định mục tiêu và đối tượng sử dụng, đến phân tích chi tiết yêu cầu chức năng và phi chức năng. Các nội dung thiết kế bao gồm kiến trúc tổng thể client-server phân lớp, giao diện người dùng, các sơ đồ UML cho chức năng tiêu biểu (Use Case, Activity, Sequence), thiết kế cơ sở dữ liệu Firestore và mô hình tổ chức mã nguồn. Kết quả phân tích trong chương này là cơ sở để triển khai chi tiết trong chương tiếp theo.

3.1 Mô tả bài toán

3.1.1 Bối cảnh thực tế

Trong môi trường học tập đại học, sinh viên thường xuyên đối mặt với bài toán tìm kiếm địa điểm học tập phù hợp. Việc xác định nơi nào có không gian yên tĩnh, wifi ổn định, ổ cắm điện hay điều hòa chủ yếu dựa vào kinh nghiệm cá nhân hoặc lời giới thiệu từ bạn bè, thiếu một nền tảng tập trung để tra cứu, đánh giá và so sánh. Bên cạnh đó, việc chia sẻ tài liệu học tập hiện chủ yếu thông qua các nhóm chat không chính thức, dẫn đến tài liệu bị phân tán, trùng lặp và khó kiểm soát chất lượng. Sinh viên cũng gặp khó khăn trong việc tìm bạn học cùng nhóm hoặc tham gia nhóm học tập một cách hiệu quả.

3.1.2 Khó khăn và hạn chế của phương pháp hiện tại

Phương pháp tìm địa điểm truyền thống dựa trên kinh nghiệm cá nhân khiến sinh viên mới gặp khó khăn trong việc tiếp cận. Việc chia sẻ tài liệu qua các nhóm chat dẫn đến tài liệu trùng lặp, không có cơ chế kiểm duyệt chất lượng, và thông tin bị phân tán qua nhiều kênh. Ngoài ra, thiếu công cụ chuyên dụng để kết nối sinh viên có cùng sở thích học tập, tạo nhóm học nhóm và trao đổi kiến thức.

3.1.3 Nhu cầu xây dựng ứng dụng

Từ thực trạng trên, nhu cầu xây dựng một nền tảng tích hợp trở nên cấp thiết. Hệ thống Sfinity ra đời nhằm giải quyết đồng thời ba bài toán: cung cấp bản đồ số hỗ trợ khám phá và đánh giá địa điểm học tập; xây dựng nền tảng quản lý và chia sẻ tài liệu

học tập có kiểm duyệt; và tạo không gian kết nối mạng xã hội cho sinh viên thông qua nhóm học tập và chat nhóm.

3.1.4 Mục tiêu hệ thống

Hệ thống hướng đến mục tiêu xây dựng ứng dụng di động cho phép người dùng khám phá địa điểm học tập trên bản đồ tương tác, tìm địa điểm gần mình qua GPS, check-in và viết đánh giá; quản lý, upload và đọc tài liệu PDF; kết bạn, tạo nhóm học tập và chat nhóm. Đồng thời, hệ thống cung cấp trang quản trị web cho phép quản trị viên kiểm soát chất lượng nội dung, quản lý người dùng và xem thống kê hệ thống.

3.1.5 Đối tượng sử dụng và phạm vi

Hệ thống phục vụ hai đối tượng chính. Người dùng (sinh viên) sử dụng ứng dụng di động để khám phá địa điểm, quản lý tài liệu, kết nối bạn bè và tham gia nhóm học tập. Quản trị viên sử dụng trang web admin để duyệt nội dung, quản lý người dùng, xem thống kê và cấu hình hệ thống. Phạm vi gồm ứng dụng Android (Flutter), web admin (React) và backend API (Node.js), sử dụng Firebase Firestore làm cơ sở dữ liệu.

3.2 Phân tích yêu cầu hệ thống

3.2.1 Yêu cầu chức năng

Hệ thống cần đáp ứng các nhóm chức năng chính sau. Thứ nhất, hệ thống phải hỗ trợ người dùng đăng ký, đăng nhập, khôi phục mật khẩu bằng OTP, đăng nhập bằng tài khoản Google, cập nhật hồ sơ cá nhân và thay đổi mật khẩu. Thứ hai, hệ thống cần cung cấp chức năng khám phá và tìm kiếm, cho phép người dùng tra cứu địa điểm học tập, tài liệu, nội dung nổi bật và các thông tin hỗ trợ khác.

Bên cạnh đó, hệ thống phải hỗ trợ quản lý địa điểm học tập, bao gồm hiển thị địa điểm trên bản đồ, xem chi tiết, tìm kiếm, lọc, check-in, đánh giá, chia sẻ, thêm mới và chỉnh sửa địa điểm. Đối với tài liệu học tập, người dùng cần có khả năng xem danh sách tài liệu, xem chi tiết, tải xuống, tạo mới, chỉnh sửa, đánh giá và đánh dấu yêu thích. Ở khía cạnh cộng đồng, hệ thống cần hỗ trợ kết bạn, quản lý danh sách bạn bè, tạo nhóm học tập, tham gia nhóm, trò chuyện nhóm theo thời gian thực và chia sẻ tài nguyên trong nhóm. Ngoài ra, hệ thống còn phải cung cấp các chức năng cá nhân hóa

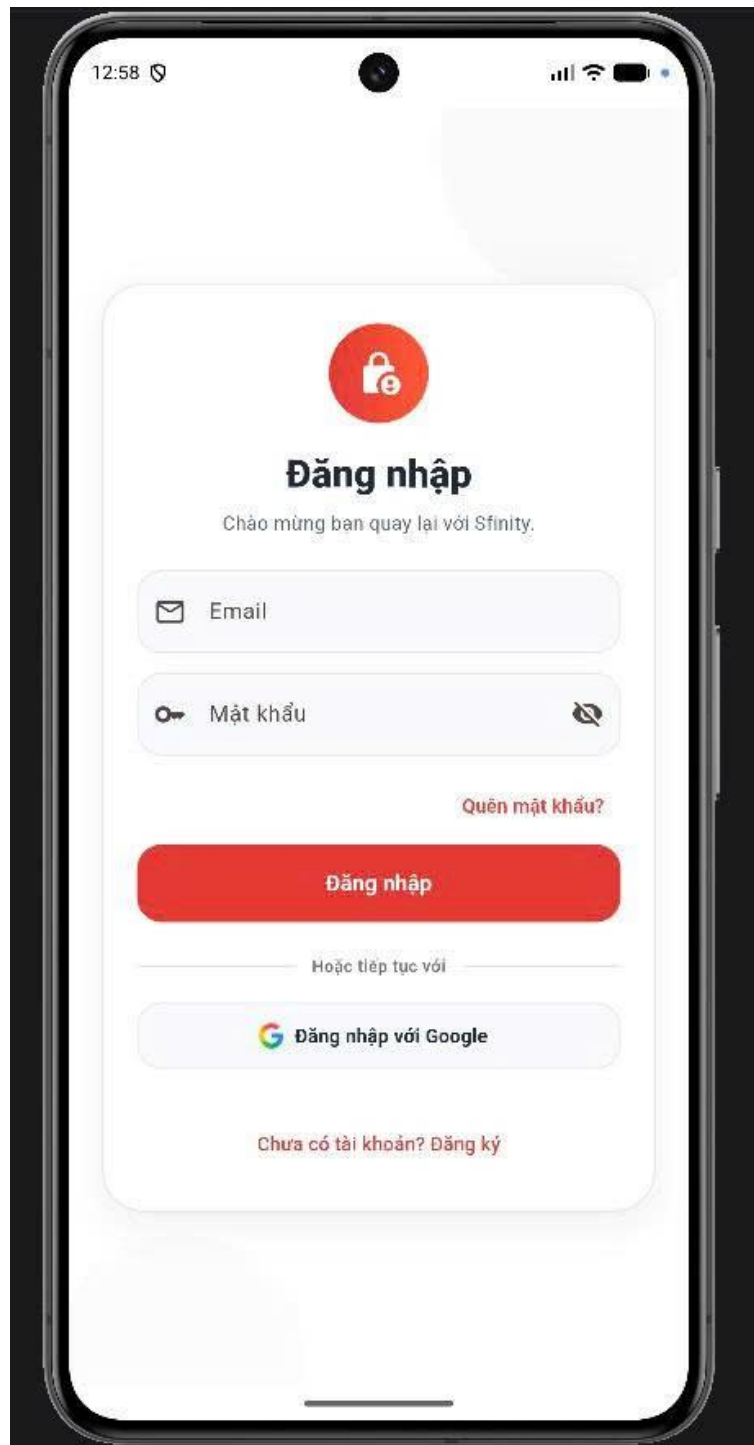
nếu quản lý yêu thích, nhận thông báo, gửi phản hồi, báo cáo vi phạm, thay đổi ngôn ngữ và giao diện hiển thị. Đối với quản trị viên, hệ thống cần hỗ trợ dashboard thống kê, quản lý người dùng, quản lý nội dung, kiểm duyệt dữ liệu và xử lý các phản hồi, báo cáo phát sinh.

3.2.2 Yêu cầu phi chức năng

Ngoài các chức năng nghiệp vụ, hệ thống cần đảm bảo một số yêu cầu phi chức năng quan trọng. Về hiệu năng, hệ thống phải có khả năng phản hồi nhanh, đặc biệt trong các thao tác tải danh sách địa điểm, tài liệu và hiển thị bản đồ. Về tính ổn định, ứng dụng cần vận hành tốt trên nền tảng di động và đảm bảo tính nhất quán trong giao tiếp giữa mobile, web-admin và backend. Về bảo mật, hệ thống cần áp dụng cơ chế xác thực và phân quyền phù hợp, đồng thời đảm bảo an toàn cho dữ liệu người dùng và nội dung được chia sẻ. Về khả năng mở rộng, kiến trúc hệ thống phải cho phép bổ sung tính năng mới mà không ảnh hưởng lớn đến các thành phần hiện có. Cuối cùng, về trải nghiệm người dùng, giao diện cần trực quan, dễ sử dụng, hỗ trợ đa ngôn ngữ và phù hợp với đặc điểm thao tác trên thiết bị di động.

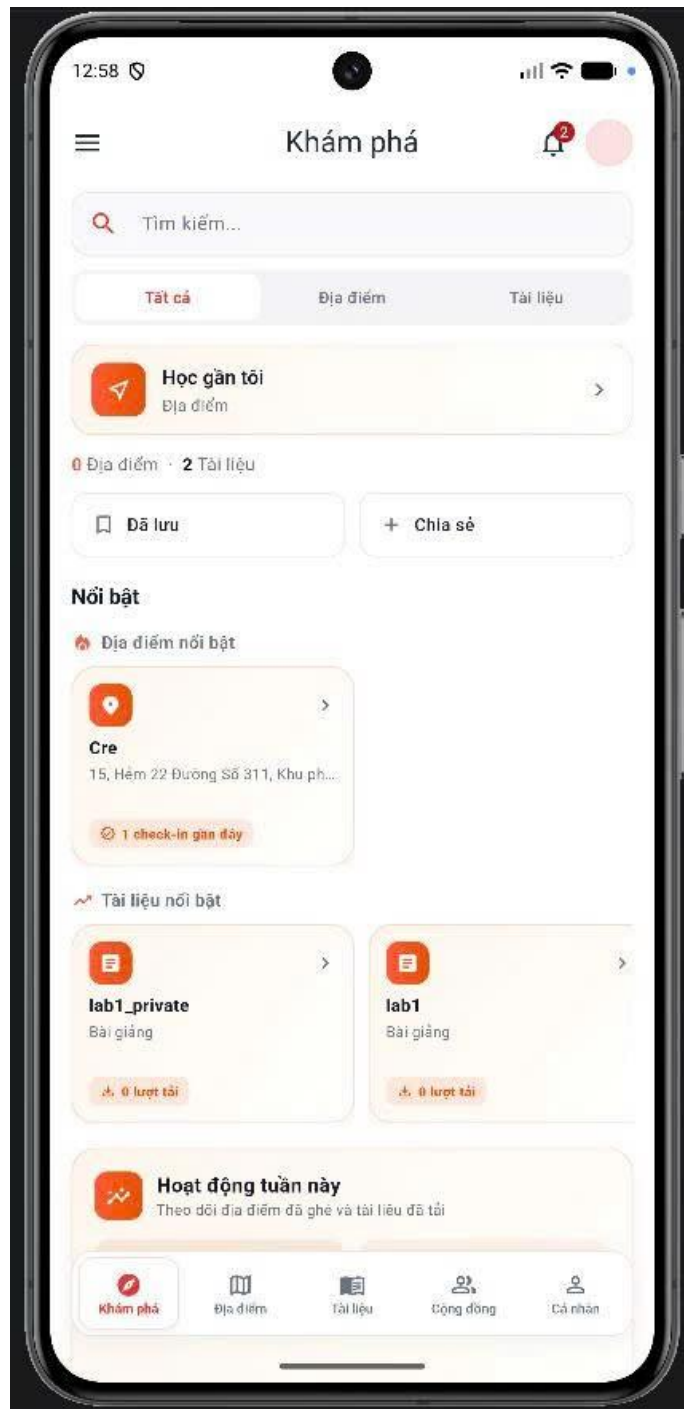
3.3 Thiết kế giao diện ứng dụng

Ứng dụng Sfinity được thiết kế theo định hướng hiện đại, trực quan và phù hợp với trải nghiệm người dùng di động. Giao diện chính được tổ chức theo mô hình điều hướng tab, giúp người dùng truy cập nhanh các nhóm chức năng trọng tâm của hệ thống. Giao diện mobile được tổ chức thành 5 tab chính: Khám phá, địa điểm, tài liệu, cộng đồng, cá nhân.



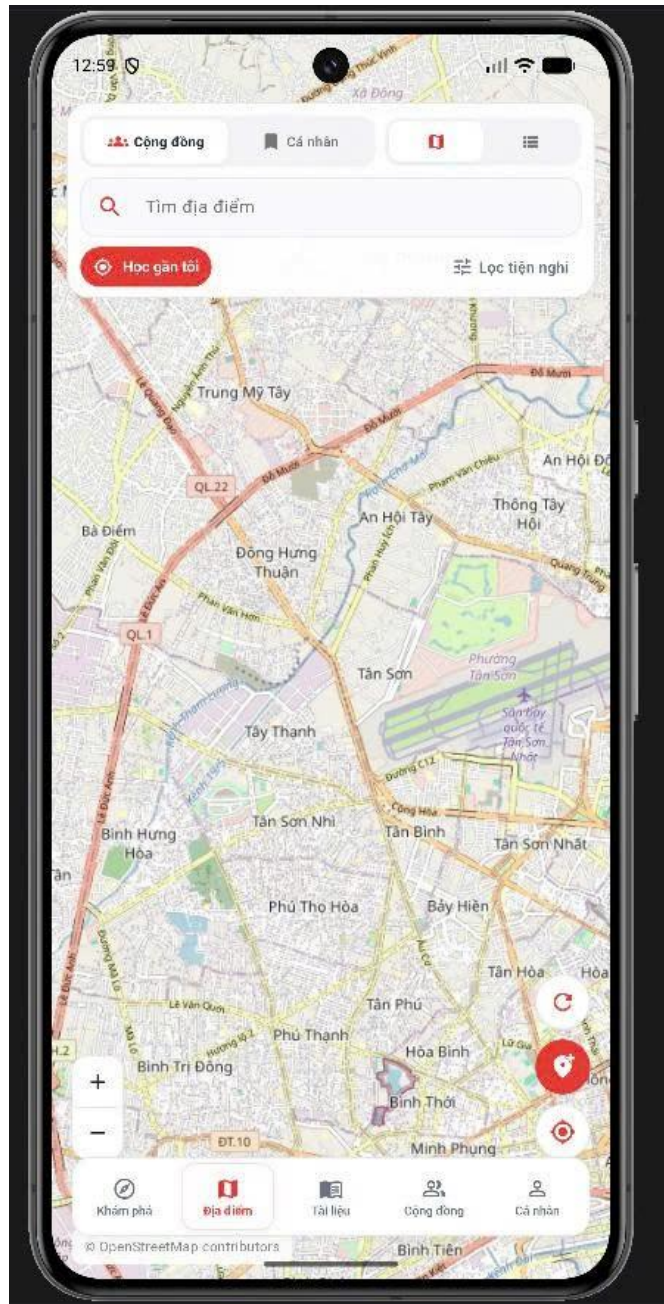
Hình 3.1 Giao diện đăng nhập/ đăng ký

Chức năng đăng nhập cho phép người dùng đăng nhập bằng email hoặc Google, hỗ trợ cho người dùng đăng ký tài khoản mới hoặc lấy lại mật khẩu bằng mã OTP nếu người dùng quên mật khẩu cũ.



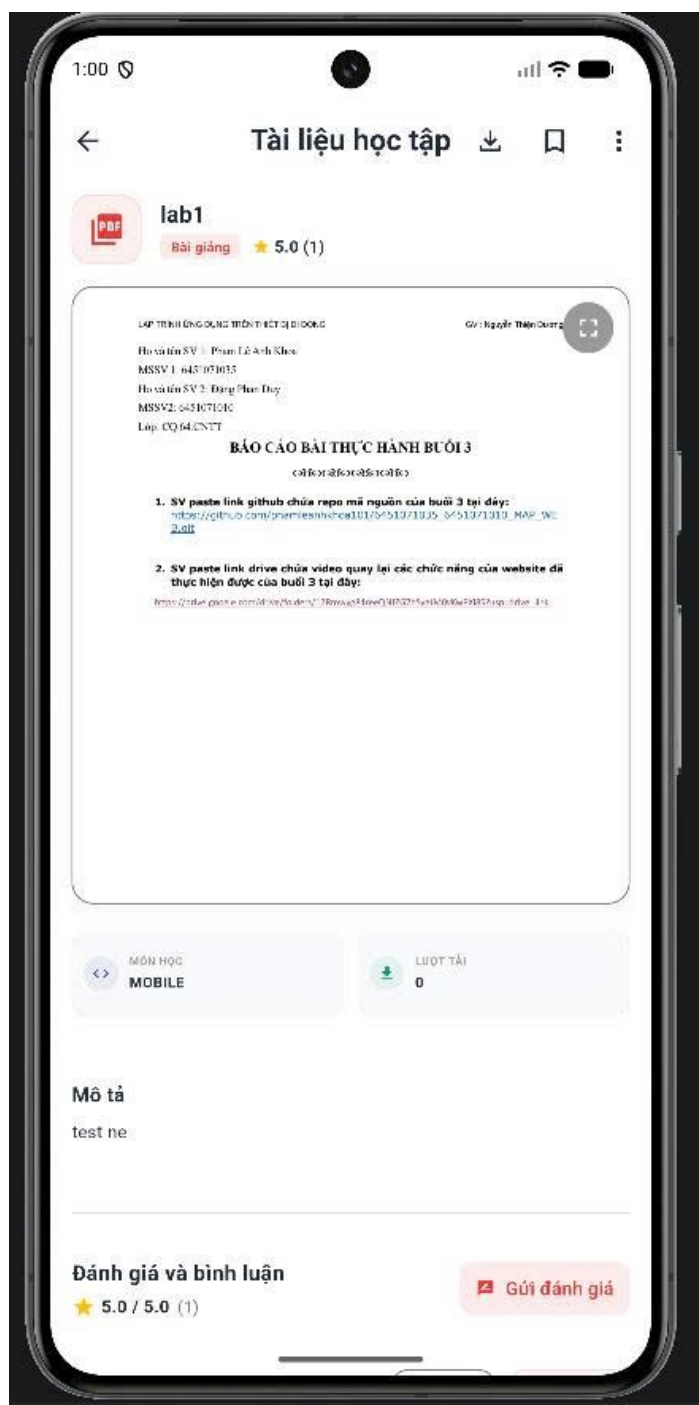
Hình 3.2 Giao diện màn hình Trang chủ

Là nơi khởi đầu để truy cập các nội dung chính, giao diện màn hình trang chủ hiển thị các nội dung nổi bật như thống kê tuần, bảng xếp hạng người dùng, đồng thời tích hợp chức năng tìm kiếm nhanh giúp người dùng truy cứu những thông tin cần thiết một cách nhanh chóng.



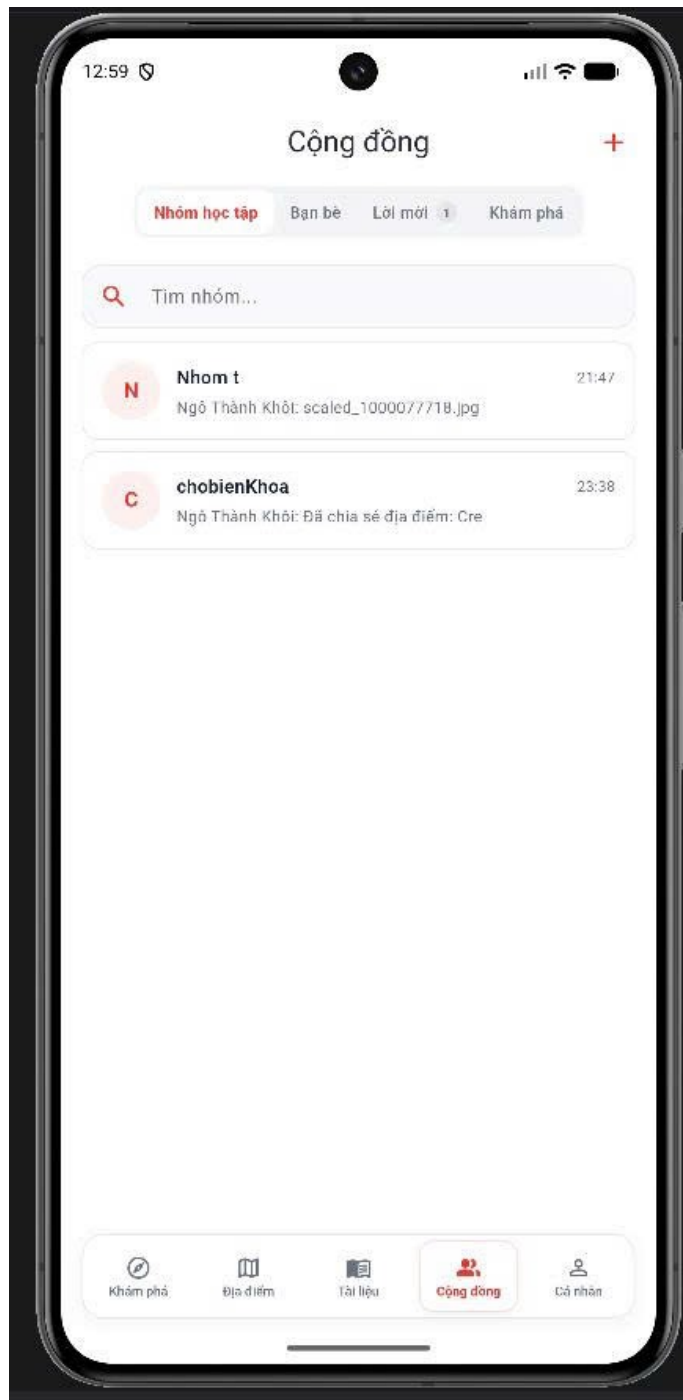
Hình 3.3 Giao diện bản đồ địa điểm

Giao diện bản đồ địa điểm cho phép hiển thị địa điểm trên nền bản đồ OpenStreetMap, người dùng có thể phóng to, thu nhỏ, lọc, tìm kiếm những địa điểm người dùng quan tâm, đồng thời hỗ trợ xem chi tiết địa điểm, check-in, đánh giá, xem ảnh, chỉ đường nhằm gửi những thông tin cần thiết cho người dùng. Có thêm chức năng “Học gần tôi” giúp hiển thị những địa điểm học tập đang ở gần địa chỉ của bạn nhất.



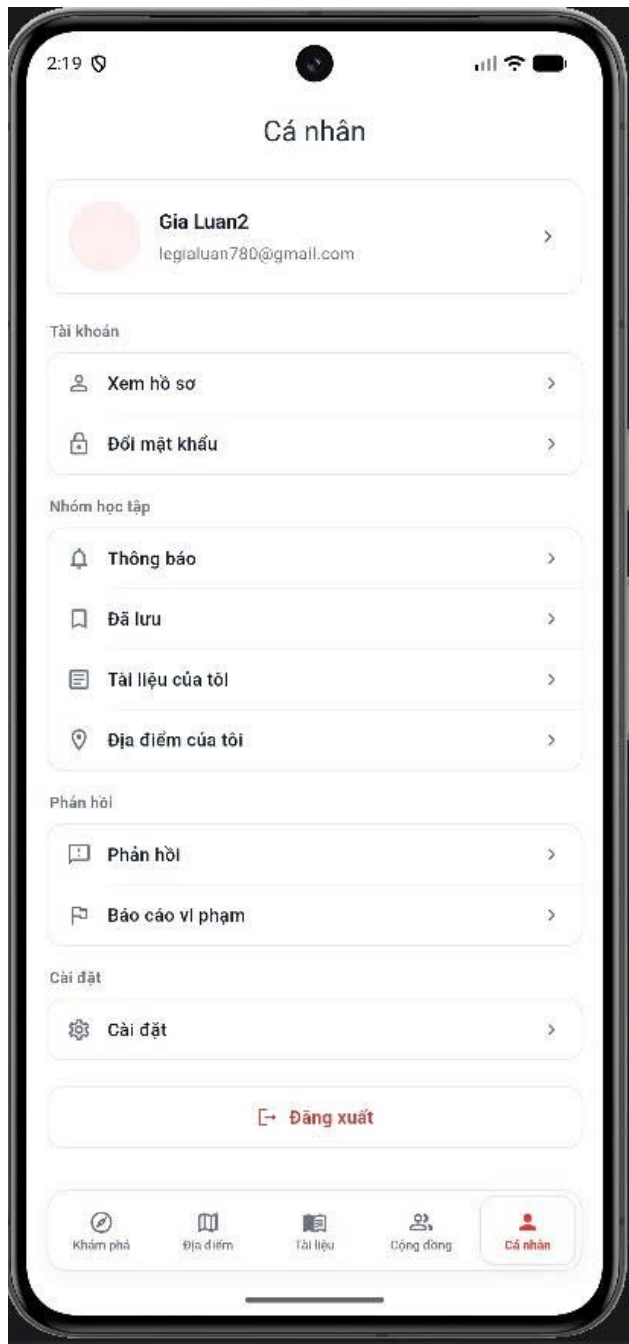
Hình 3.4 Giao diện tài liệu

Giao diện tài liệu cho phép hiển thị danh sách tài liệu công khai, hỗ trợ người dùng xem chi tiết, tải xuống, đánh dấu yêu thích và đánh giá tài liệu. Ngoài ra, giao diện còn cho phép người dùng tạo và chỉnh sửa tài liệu của riêng mình, chia sẻ cho mọi người cùng xem.



Hình 3.5 Giao diện cộng đồng

Giao diện cộng đồng hỗ trợ người dùng hiển thị danh sách bạn bè và nhóm học tập, đồng thời giúp người dùng có thể trò chuyện nhóm, chia sẻ tài liệu, ảnh, tệp và vị trí cho nhau. Một chức năng đặc biệt đó là hỗ trợ người dùng xem vị trí của các thành viên nhóm trên bản đồ.



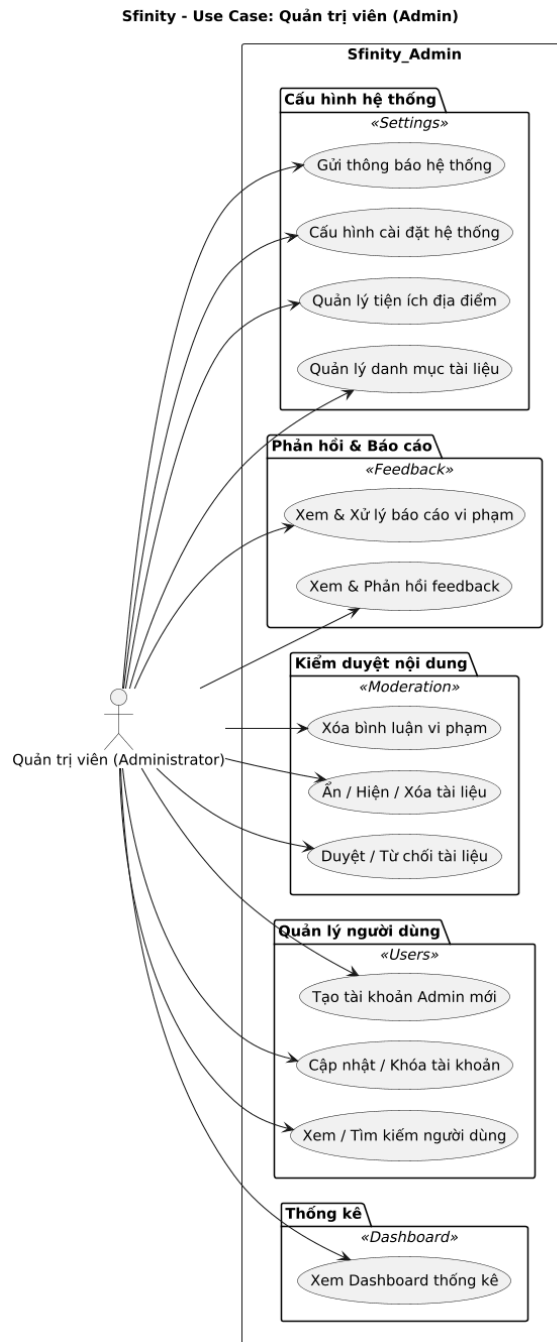
Hình 3.6 Giao diện hồ sơ và cài đặt

Giao diện hồ sơ cho phép hiển thị thông tin cá nhân của người dùng. Ngoài ra, còn hỗ trợ cho người dùng chỉnh sửa hồ sơ, đổi mật khẩu, quản lý mục yêu thích, phản hồi, báo cáo và thông báo.

Giao diện cài đặt có những chức năng thú vị như thay đổi ngôn ngữ phù hợp cho người dùng là người nước ngoài, giao diện sáng tối phù hợp cho hoàn cảnh.

3.4 Phân tích và thiết kế hệ thống

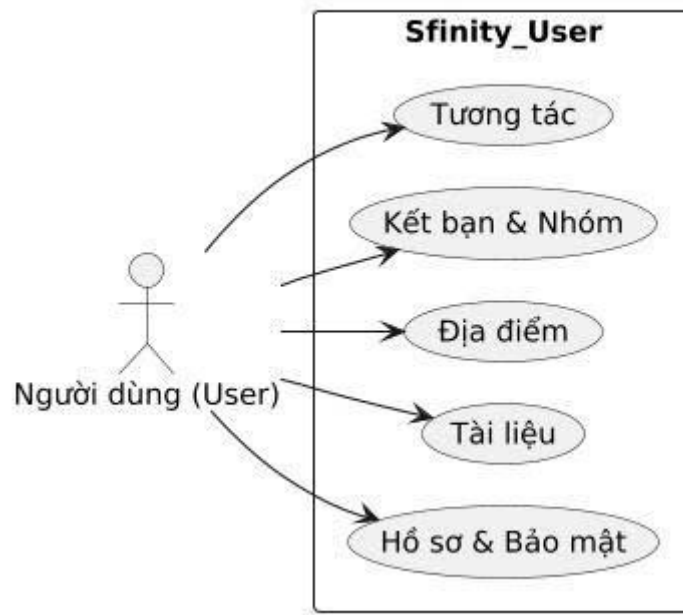
3.4.1 Sơ đồ Use Cases



Hình 3.7 Biểu đồ Use Cases cho vai trò quản trị viên

Quản trị viên có thể đăng nhập, theo dõi số liệu thống kê tổng quan, quản lý người dùng, quản lý tài liệu và địa điểm học tập.

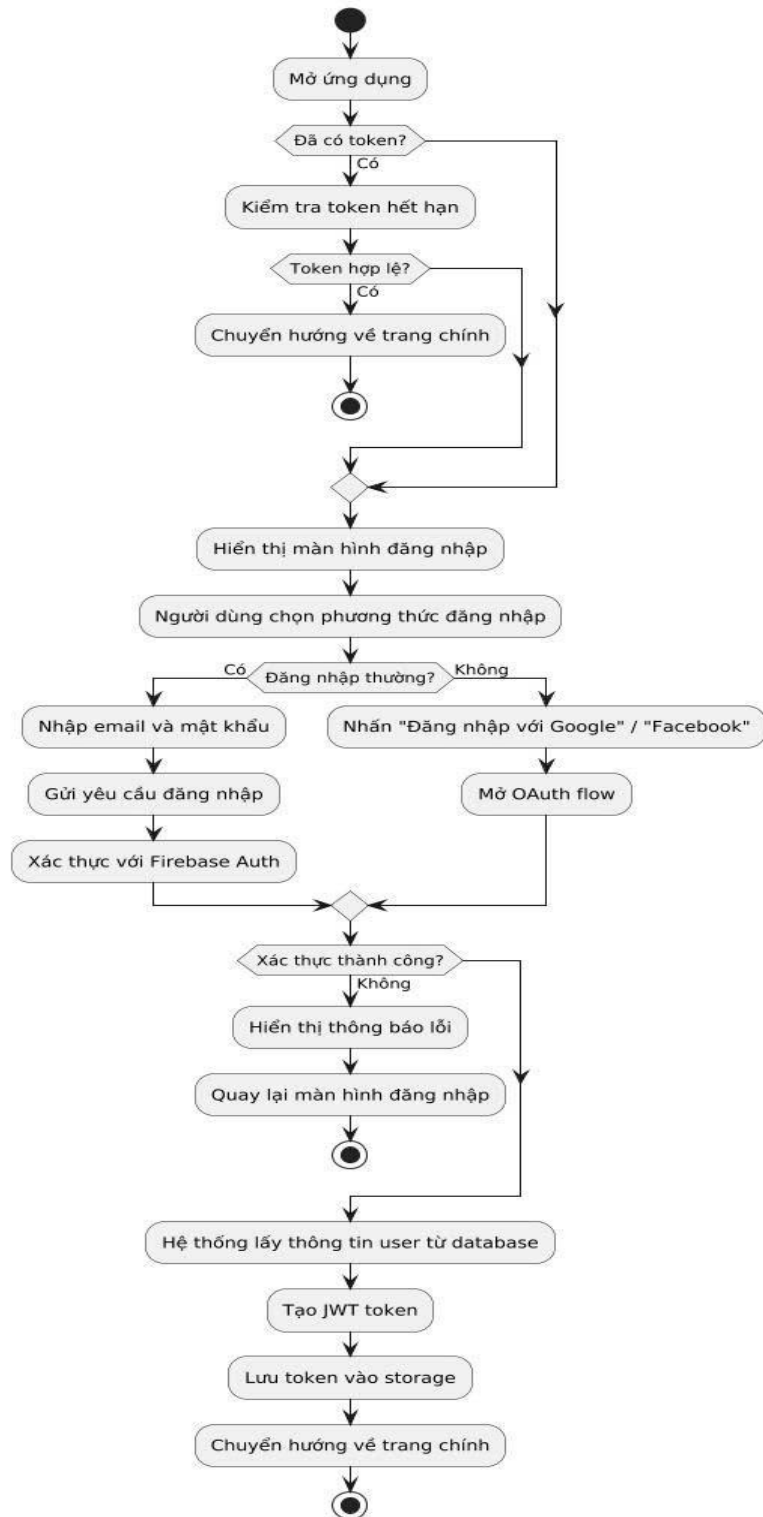
Sfinity - Use Case Tổng quan Người dùng



Hình 3.8 Biểu đồ Use Cases cho người dùng

Người dùng có thể đăng ký, đăng nhập, quản lý hồ sơ cá nhân, tìm kiếm và khám phá địa điểm học tập, tra cứu và chia sẻ tài liệu, kết bạn, tham gia nhóm học tập, trò chuyện nhóm, nhận thông báo, gửi phản hồi và báo cáo vi phạm.

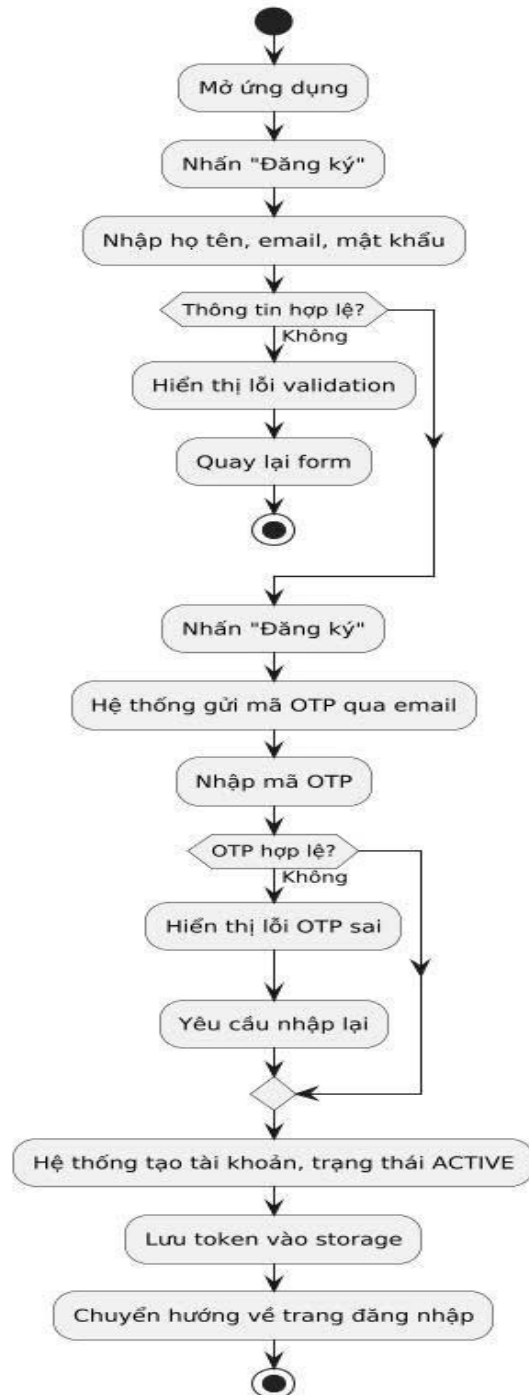
3.4.2 Biểu đồ hoạt động (Activity Diagram)



Hình 3.9 Biểu đồ hoạt động của đăng nhập

Sơ đồ hoạt động trên mô tả quy trình đăng nhập của người dùng trong hệ thống Sfinity, bắt đầu từ bước kiểm tra token đã lưu đến lựa chọn phương thức đăng nhập thông thường hoặc đăng nhập qua Google/Facebook. Sau khi xác thực thành công, hệ

thông lấy thông tin người dùng, tạo và lưu JWT token, rồi chuyển người dùng vào trang chính; nếu xác thực thất bại, hệ thống sẽ thông báo lỗi và quay lại màn hình đăng nhập.



Hình 3.10 Biểu đồ hoạt động của đăng ký

Sơ đồ hoạt động này mô tả quy trình đăng ký tài khoản của người dùng, từ bước nhập thông tin cá nhân, kiểm tra tính hợp lệ của dữ liệu đến xác thực OTP được gửi

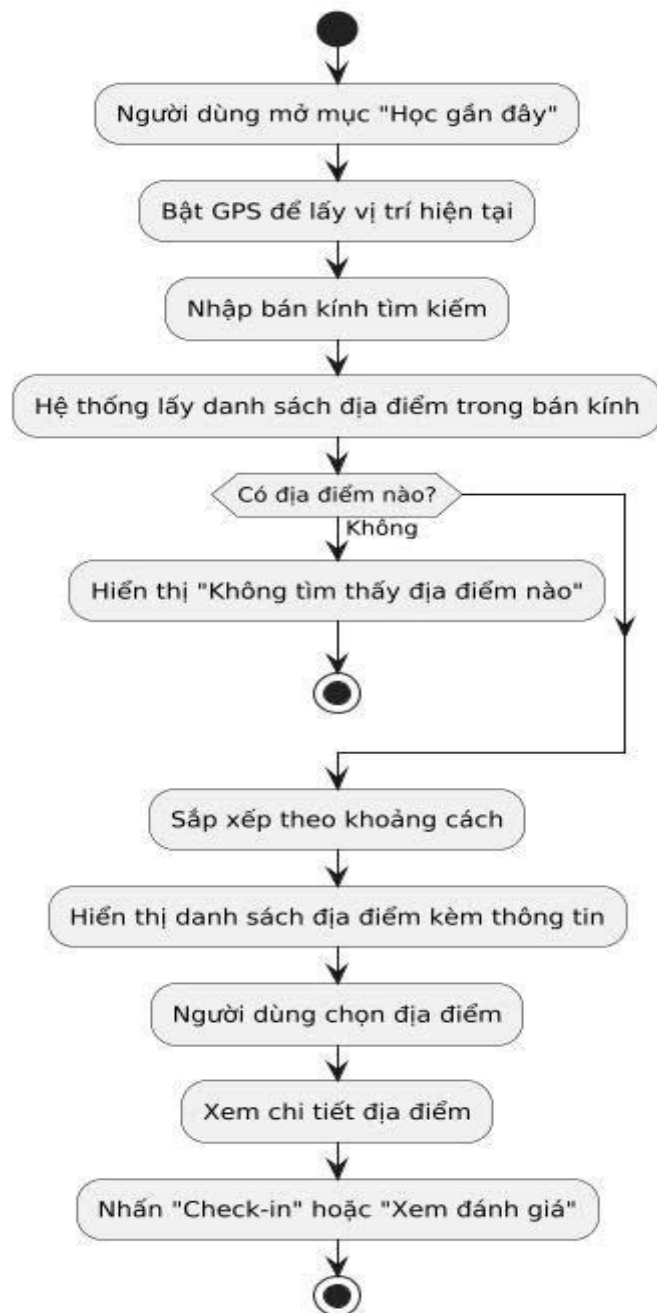
qua email. Khi thông tin và mã OTP đều hợp lệ, hệ thống tạo tài khoản ở trạng thái hoạt động, lưu token và chuyển người dùng về màn hình đăng nhập.



Hình 3.11 Biểu đồ hoạt động checking địa điểm học tập

Sơ đồ hoạt động checking địa điểm học tập mô tả quy trình người dùng thực hiện check-in tại một địa điểm học tập, bắt đầu từ việc mở bản đồ, lấy vị trí hiện tại, chọn địa điểm và gửi yêu cầu check-in. Hệ thống sẽ kiểm tra khoảng cách GPS, nếu người

dùng ở trong phạm vi cho phép thì tạo bản ghi check-in và cập nhật thông tin địa điểm; ngược lại sẽ hiển thị thông báo lỗi.



Hình 3.12 Biểu đồ hoạt động điểm học gần tôi

Sơ đồ hoạt động trên mô tả quy trình người dùng tìm kiếm địa điểm học tập gần mình dựa trên vị trí GPS và bán kính mong muốn. Hệ thống sẽ lọc, sắp xếp các địa điểm theo khoảng cách rồi hiển thị danh sách để người dùng xem chi tiết, check-in hoặc xem đánh giá.



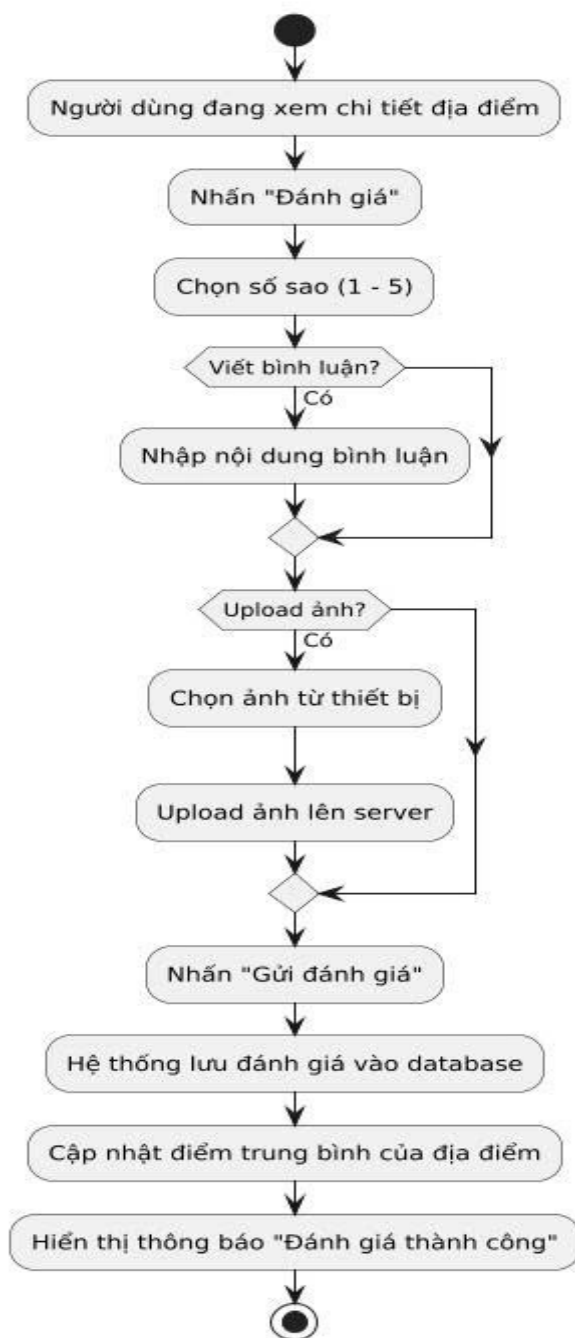
Hình 3.13 Biểu đồ hoạt động tạo tài liệu

Sơ đồ hoạt động này mô tả quy trình người dùng tạo tài liệu mới bằng cách nhập thông tin, có thể đính kèm file và chọn lưu nháp hoặc gửi duyệt. Nếu gửi duyệt, hệ thống sẽ tạo bản ghi ở trạng thái PENDING, gửi thông báo cho Admin và phản hồi lại cho người dùng rằng tài liệu đã được gửi xét duyệt.



Hình 3.14 Biểu đồ hoạt động duyệt tài liệu

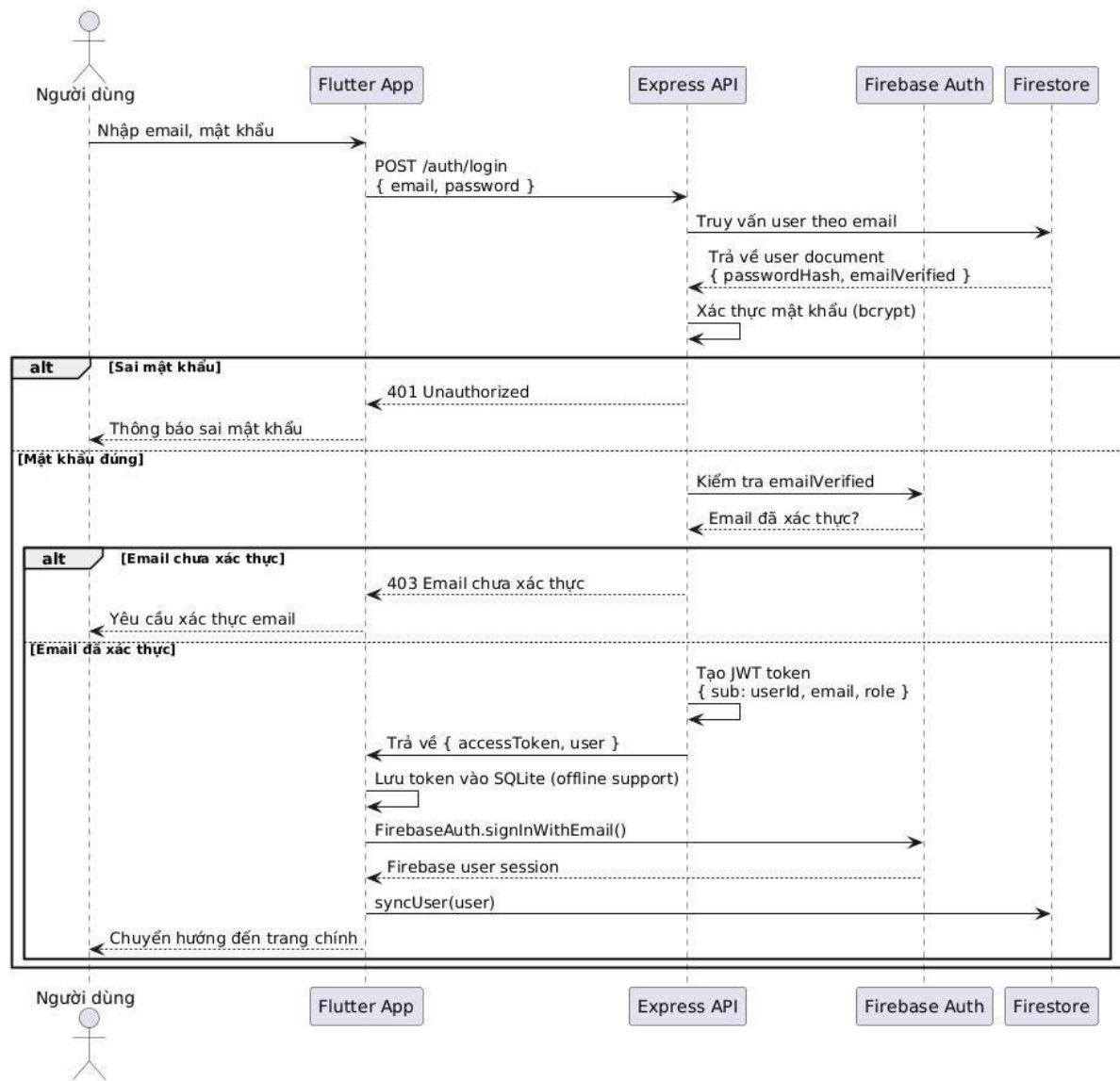
Sơ đồ hoạt động trên mô tả quy trình Admin duyệt tài liệu do người dùng gửi lên, bắt đầu từ việc xem danh sách tài liệu chờ duyệt và kiểm tra chi tiết nội dung. Nếu tài liệu phù hợp, hệ thống cập nhật trạng thái PUBLISHED; nếu không, Admin nhập lý do từ chối và hệ thống cập nhật trạng thái REJECTED, đồng thời gửi thông báo kết quả cho người tạo tài liệu.



Hình 3.15 Biểu đồ hoạt động đánh giá địa điểm

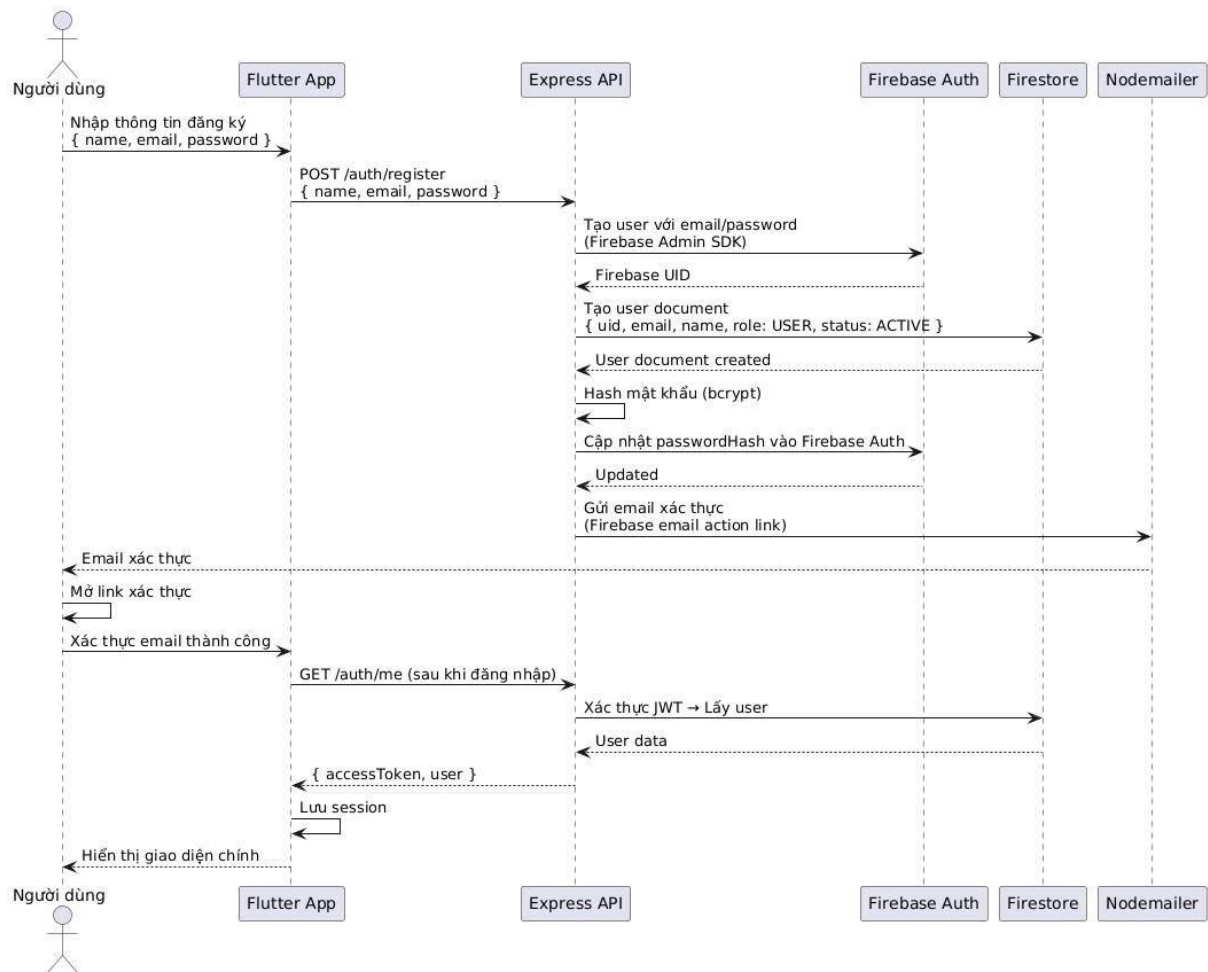
Sơ đồ hoạt động trên mô tả quy trình người dùng đánh giá một địa điểm sau khi xem chi tiết. Người dùng chọn số sao, có thể thêm bình luận và tải ảnh minh họa, sau đó gửi đánh giá để hệ thống lưu vào cơ sở dữ liệu, cập nhật điểm trung bình của địa điểm và thông báo đánh giá thành công.

3.4.3 Biểu đồ tuần tự (Sequence Diagram)



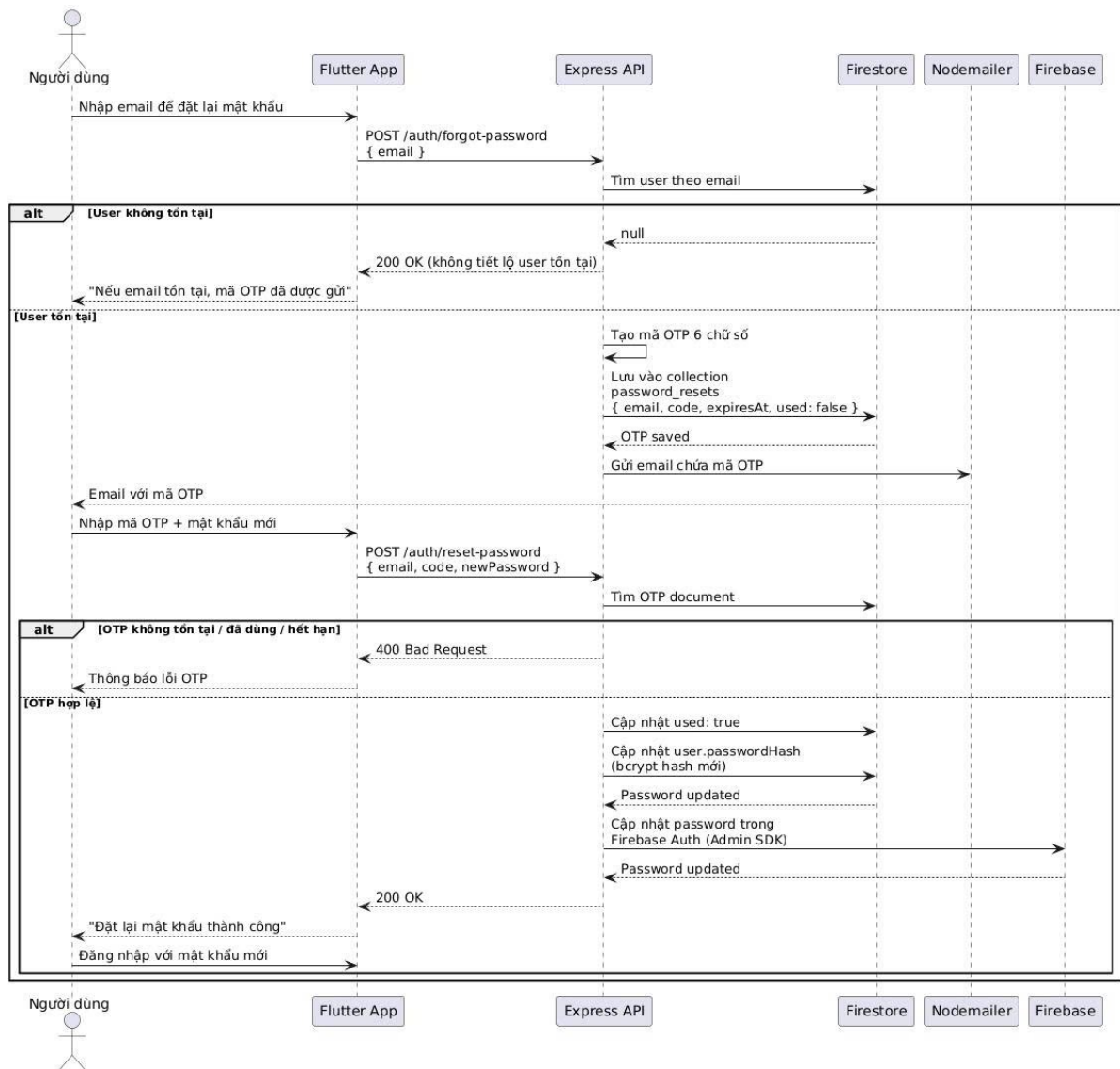
Hình 3.16 Biểu đồ tuần tự đăng nhập

Biểu đồ tuần tự trên mô tả quy trình đăng nhập: người dùng nhập email và mật khẩu, ứng dụng gửi yêu cầu đến Express API để kiểm tra tài khoản, xác thực mật khẩu và trạng thái emailVerified. Nếu hợp lệ, hệ thống tạo JWT, lưu token để hỗ trợ offline, đồng bộ phiên Firebase và chuyển người dùng vào trang chính; ngược lại sẽ trả lỗi tương ứng.



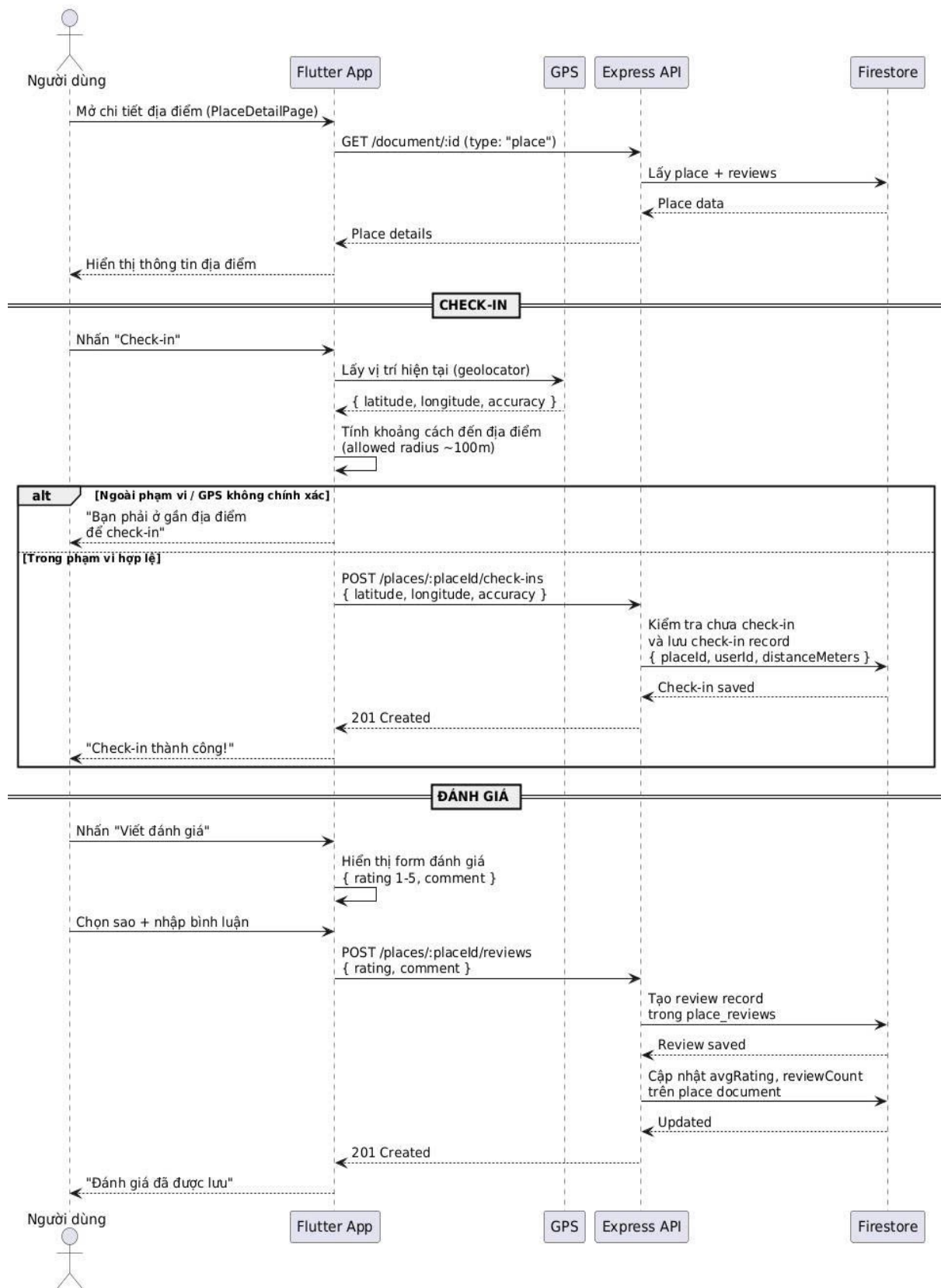
Hình 3.17 Biểu đồ tuần tự đăng ký

Biểu đồ tuần tự trên mô tả quy trình đăng ký tài khoản: người dùng nhập thông tin, ứng dụng gửi yêu cầu đến Express API để tạo tài khoản trên Firebase Auth, lưu hồ sơ người dùng vào Firestore và gửi email xác thực. Sau khi người dùng xác thực email thành công, hệ thống lấy thông tin tài khoản, tạo session và đưa người dùng vào giao diện chính.



Hình 3.18 Biểu đồ tuần tự quên mật khẩu

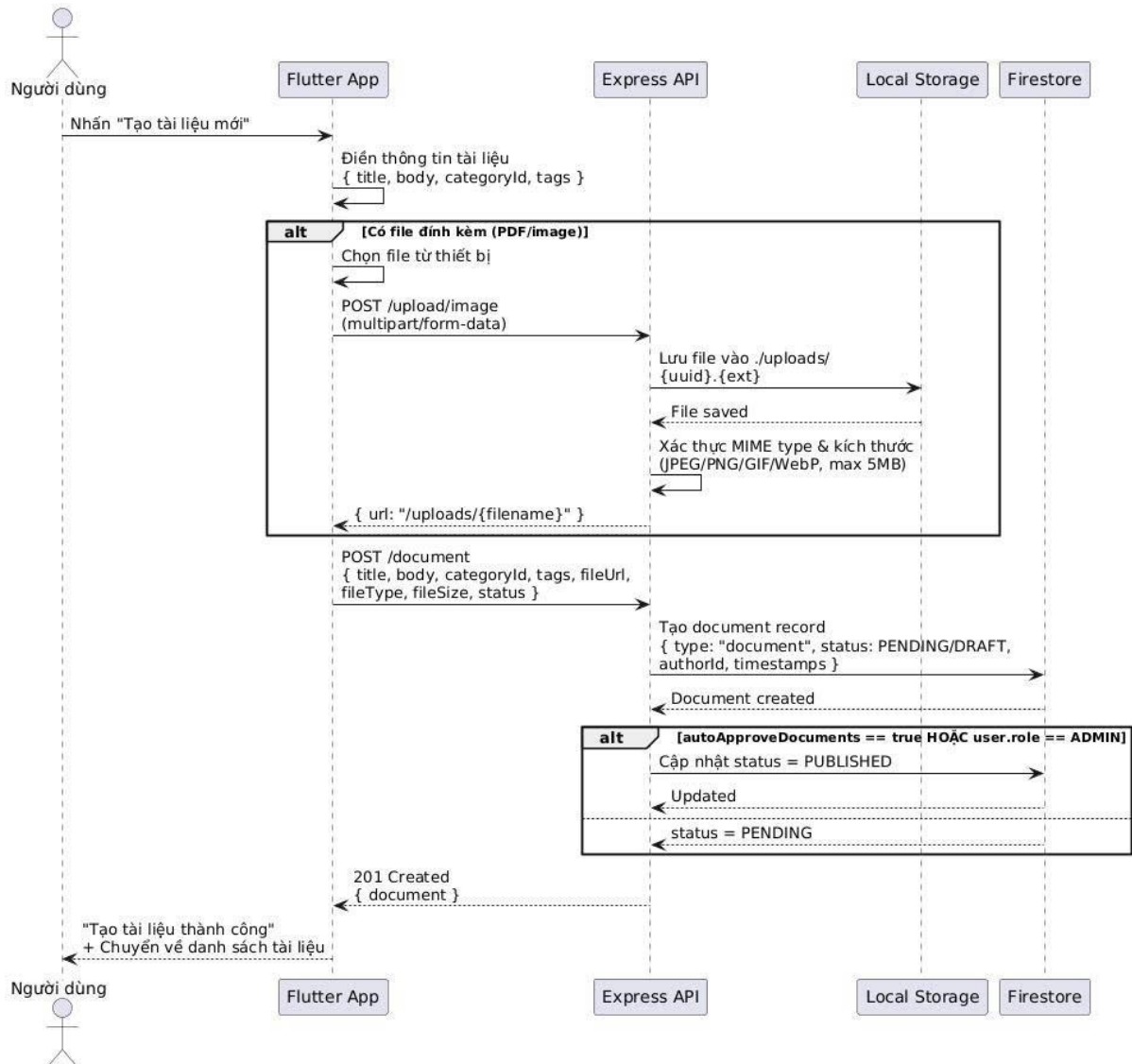
Biểu đồ tuần tự trên mô tả quy trình quên mật khẩu: người dùng nhập email, hệ thống kiểm tra tài khoản và nếu hợp lệ sẽ tạo mã OTP, lưu thời hạn sử dụng rồi gửi qua email. Sau đó người dùng nhập OTP cùng mật khẩu mới để xác thực; nếu mã đúng và còn hạn, hệ thống cập nhật mật khẩu trong Firestore và Firebase Auth rồi thông báo đặt lại mật khẩu thành công.



Hình 3.19 Biểu đồ tuần tự checkin và đánh giá

Biểu đồ tuần tự trên mô tả quy trình xem chi tiết địa điểm, check-in và đánh giá: ứng dụng lấy thông tin địa điểm từ hệ thống, sau đó khi người dùng check-in sẽ dùng

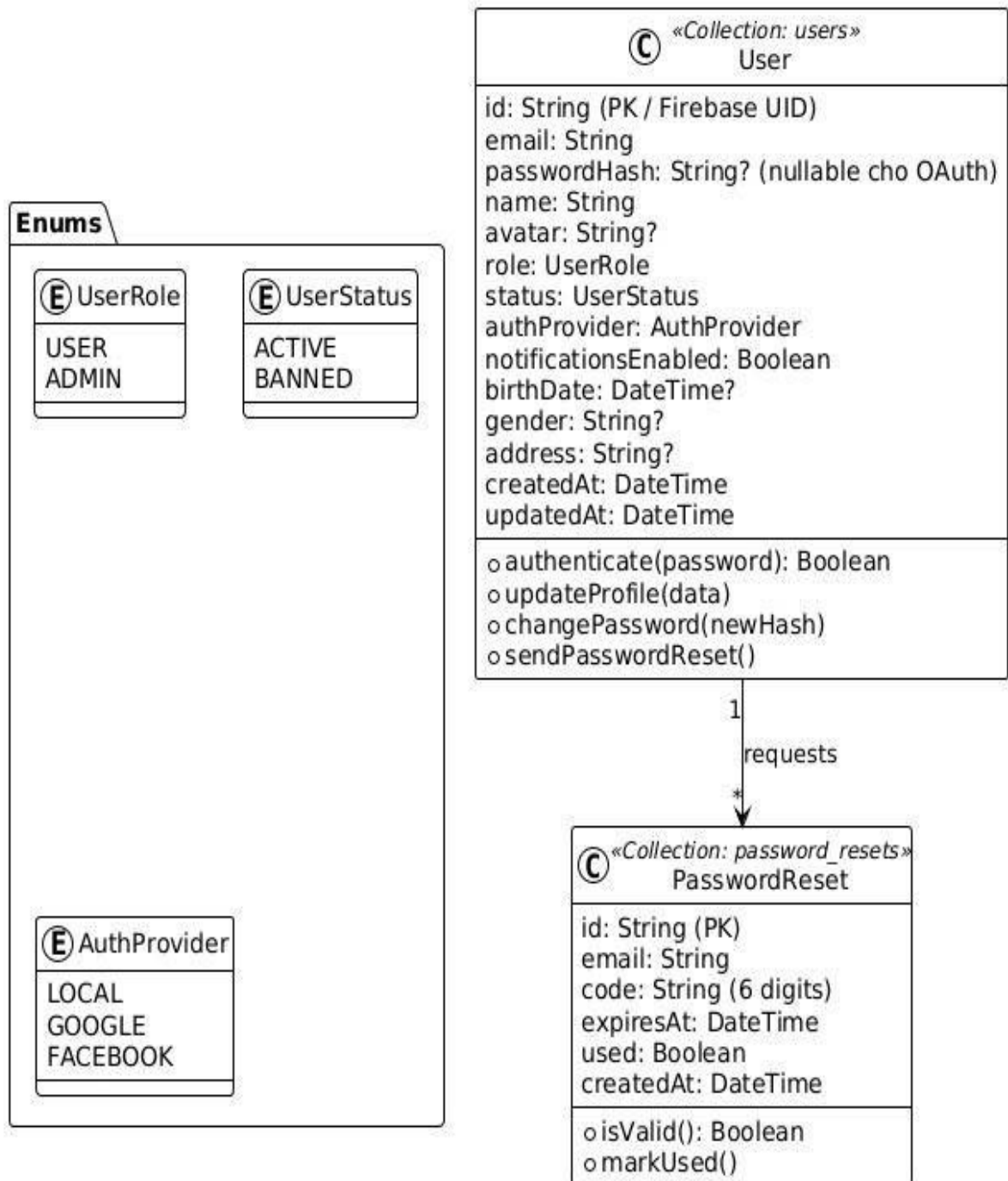
GPS để xác định vị trí và chỉ cho phép ghi nhận nếu đang ở trong phạm vi hợp lệ. Tiếp theo, người dùng có thể gửi đánh giá kèm số sao và bình luận, hệ thống lưu review vào Firestore đồng thời cập nhật điểm trung bình và số lượt đánh giá của địa điểm.



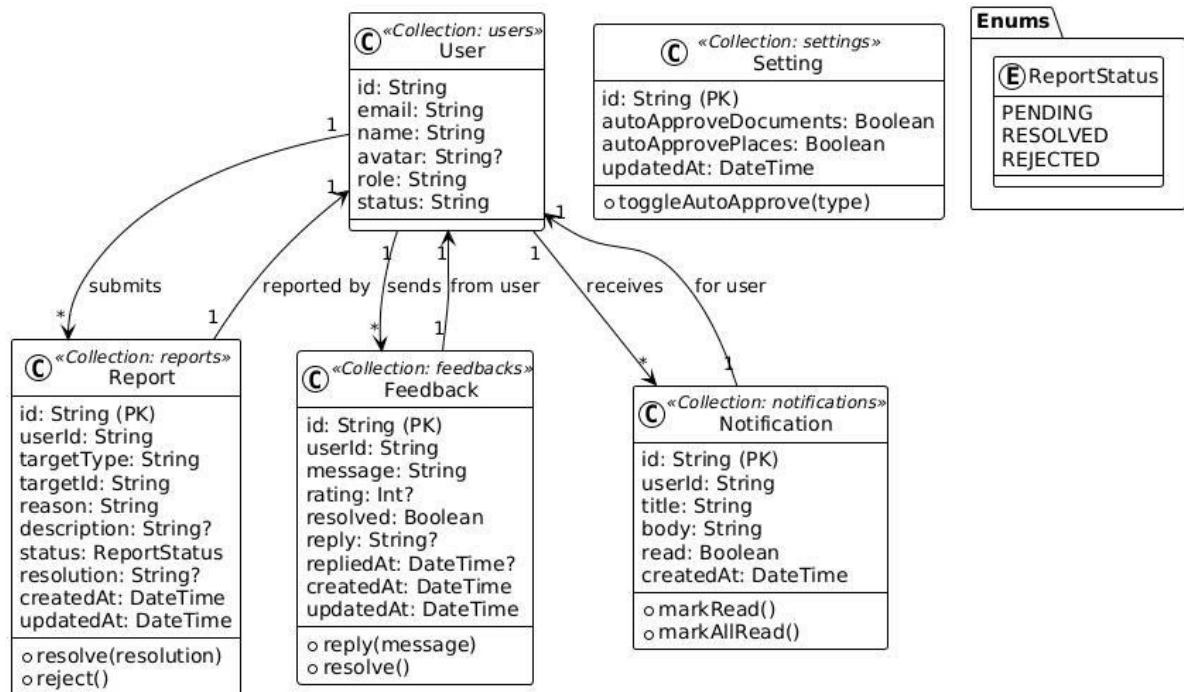
Hình 3.20 Biểu đồ tuần tự tạo tài liệu

Biểu đồ tuần tự trên mô tả quy trình tạo tài liệu mới: người dùng nhập thông tin tài liệu, có thể tải kèm file PDF hoặc ảnh để hệ thống kiểm tra định dạng, kích thước và lưu file vào bộ nhớ cục bộ trước khi tạo bản ghi tài liệu trong Firestore. Sau đó hệ thống thiết lập trạng thái tài liệu là PUBLISHED nếu được tự duyệt hoặc người tạo là admin, ngược lại sẽ để PENDING, rồi trả kết quả thành công và chuyển người dùng về danh sách tài liệu.

3.4.4 Biểu đồ lớp



Hình 3.21 Biểu đồ lớp cho tính năng xác thực



Hình 3.22 Biểu đồ lớp cho tính năng quản lý

3.5 Thiết kế cơ sở dữ liệu

Ứng dụng Sfinity sử dụng mô hình cơ sở dữ liệu kết hợp giữa Firebase Firestore, Firebase Storage và SQLite. Trong đó, Firestore đảm nhiệm lưu trữ dữ liệu đám mây theo mô hình NoSQL để phục vụ các chức năng cần đồng bộ thời gian thực như hồ sơ người dùng và tin nhắn nhóm. Firebase Storage được dùng để lưu trữ các tệp và hình ảnh do người dùng tải lên. Bên cạnh đó, SQLite được sử dụng như cơ sở dữ liệu cục bộ trên thiết bị nhằm lưu thông tin phiên đăng nhập và các thiết lập ứng dụng, giúp tăng tốc truy cập và hỗ trợ duy trì trạng thái khi sử dụng ứng dụng.

Các bảng / collection dữ liệu chính:

- Collection users: Lưu thông tin hồ sơ người dùng được đồng bộ từ Firebase Authentication. Các thuộc tính chính bao gồm:
 - uid: mã người dùng
 - email: email đăng nhập
 - displayName: tên hiển thị
 - photoURL: ảnh đại diện

- provider: nhà cung cấp đăng nhập
- birthDate: ngày sinh
- gender: giới tính
- address: địa chỉ
- createdAt: thời điểm tạo
- updatedAt: thời điểm cập nhật
- Collection groups: Lưu thông tin nhóm học tập/cộng đồng. Các thuộc tính chính:
 - id: mã nhóm
 - name: tên nhóm
 - description: mô tả
 - avatarUrl: ảnh đại diện nhóm
 - isPublic
 - creatorId: thứ tự tạo
 - createdAt: thời điểm tạo
 - memberCount: Số thành viên
 - autoApprove
 - myRole
 - myStatus
- Subcollection groups/{groupId}/messages: Lưu tin nhắn trao đổi trong từng nhóm theo thời gian thực. Các thuộc tính chính:
 - senderId: mã người gửi
 - senderName: tên người gửi
 - senderAvatar: ảnh đại diện người gửi
 - text: nội dung tin nhắn

- type: loại tin nhắn (text, document, image, file, location)
- sharedDocumentId: mã tài liệu được chia sẻ
- sharedDocumentTitle: tên tài liệu được chia sẻ
- sharedPlaceId: mã địa điểm được chia sẻ
- fileUrl: đường dẫn file/ảnh
- fileName: tên file
- fileSize: kích thước file
- createdAt: thời gian gửi
- reactions: cảm xúc theo từng người dùng
- isDeleted: trạng thái thu hồi tin nhắn

3.6 Thiết kế kiến trúc hệ thống

Hệ thống Sfinity được xây dựng theo mô hình client–server kết hợp dịch vụ đám mây. Trong đó, ứng dụng di động phát triển bằng Flutter đóng vai trò client, chịu trách nhiệm hiển thị giao diện, tiếp nhận thao tác người dùng và gửi yêu cầu xử lý dữ liệu. Phía server/backend đảm nhiệm các nghiệp vụ chính như xác thực, quản lý dữ liệu tài liệu, địa điểm, nhóm học tập và các chức năng liên quan. Bên cạnh đó, hệ thống còn tích hợp Firebase để hỗ trợ một số dịch vụ như đăng nhập, đồng bộ dữ liệu thời gian thực, trò chuyện nhóm và lưu trữ tệp, đồng thời sử dụng SQLite để lưu dữ liệu cục bộ trên thiết bị.

Về cơ chế giao tiếp, ứng dụng gửi yêu cầu từ giao diện người dùng đến các lớp xử lý trung gian trong client, sau đó truy xuất dữ liệu thông qua repository. Tầng repository đóng vai trò kết nối với nhiều nguồn dữ liệu khác nhau như backend API, Firebase Firestore, Firebase Storage và SQLite. Dữ liệu nhận được sẽ được ánh xạ sang các model của ứng dụng trước khi trả lại cho giao diện để hiển thị. Cách tổ chức này giúp giảm sự phụ thuộc trực tiếp giữa giao diện và nguồn dữ liệu, đồng thời làm cho hệ thống dễ bảo trì và mở rộng.

Xét về mô hình tổ chức mã nguồn, project được phát triển theo hướng feature-based kết hợp với kiến trúc phân lớp. Mã nguồn được chia theo từng chức năng như đăng nhập, tài liệu, địa điểm, nhóm học tập, kết bạn và hồ sơ người dùng. Trong mỗi chức năng, các thành phần tiếp tục được tách thành các nhóm như giao diện, controller, model, service và repository. Nếu đối chiếu với các mô hình kiến trúc phổ biến, hệ thống có đặc điểm gần với MVVM, trong đó giao diện đóng vai trò View, các lớp controller quản lý trạng thái giữ vai trò tương tự ViewModel, còn model, repository và service tạo thành tầng dữ liệu và nghiệp vụ. Cách tổ chức này phù hợp với ứng dụng di động có quy mô vừa, giúp việc phát triển, kiểm thử và nâng cấp hệ thống được thuận lợi hơn.

Tóm lại, kiến trúc hệ thống của Sfinity là sự kết hợp giữa client-server, dịch vụ đám mây và lưu trữ cục bộ, được tổ chức mã nguồn theo hướng phân tách rõ ràng giữa giao diện, xử lý trạng thái và dữ liệu. Đây là nền tảng quan trọng giúp hệ thống hoạt động ổn định, đáp ứng yêu cầu hiện tại và sẵn sàng mở rộng trong các giai đoạn phát triển tiếp theo.

Kết luận chương: Trong chương này, đề tài đã trình bày các nội dung thiết kế quan trọng của hệ thống, bao gồm thiết kế cơ sở dữ liệu, các thực thể dữ liệu chính, mối quan hệ giữa chúng và kiến trúc tổng thể của ứng dụng. Đồng thời, mô hình hoạt động của hệ thống theo hướng client-server, cách ứng dụng giao tiếp với backend, Firebase và SQLite cũng đã được làm rõ.

CHƯƠNG 4 CÀI ĐẶT THỰC NGHIỆM VÀ ĐÁNH GIÁ

Tóm tắt chương: Chương này trình bày quá trình cài đặt và hiện thực hệ thống Sfinity trên cơ sở các nội dung phân tích và thiết kế đã được xây dựng ở chương trước. Nội dung chương tập trung mô tả cấu trúc project, cách tổ chức mã nguồn, các module chức năng chính của ứng dụng mobile và backend, cùng cơ chế xử lý dữ liệu giữa Flutter, REST API, Firebase Firestore, Firebase Storage và SQLite. Bên cạnh đó, chương cũng trình bày việc ứng dụng công cụ AI trong quá trình phát triển, các kịch bản thực nghiệm nhằm kiểm tra hoạt động của hệ thống, cũng như phần đánh giá kết quả đạt được, những ưu điểm, hạn chế và khó khăn trong quá trình triển khai.

4.1 Cài đặt hệ thống

4.1.1 Môi trường và công nghệ cài đặt

Hệ thống Sfinity được xây dựng theo mô hình monorepo, gồm ba thành phần chính là ứng dụng di động mobile, hệ thống quản trị web-admin và dịch vụ backend backend. Trong phạm vi chương này, phần cài đặt thực nghiệm tập trung chủ yếu vào ứng dụng mobile và backend, do đây là hai thành phần trực tiếp tham gia cung cấp chức năng cho người dùng cuối.

Ứng dụng mobile được phát triển bằng Flutter, sử dụng ngôn ngữ Dart và chạy trên nhiều nền tảng như Android, iOS và Web. Dự án tích hợp nhiều thư viện quan trọng như dio để gọi API, go_router để định tuyến màn hình, firebase_core, firebase_auth, cloud_firestore, firebase_storage để kết nối hệ sinh thái Firebase, sqflite để lưu trữ cục bộ, flutter_map và geolocator để xử lý bản đồ và vị trí người dùng.

Phía backend được xây dựng bằng Node.js, Express và TypeScript, cung cấp REST API dùng chung cho mobile và web-admin. Backend hỗ trợ xác thực bằng JWT, phân quyền người dùng theo vai trò, tích hợp Swagger để kiểm thử API, sử dụng Firestore làm kho dữ liệu chính và Prisma với SQLite cho một phần metadata nội bộ.

Nhìn chung, việc lựa chọn Flutter kết hợp Firebase và backend Express giúp hệ thống vừa có khả năng phát triển giao diện nhanh, vừa đảm bảo tính linh hoạt trong xử lý nghiệp vụ, đồng bộ dữ liệu và khả năng mở rộng.

4.1.2 Cấu trúc project và tổ chức source code

Qua khảo sát mã nguồn, project được tổ chức thành các thư mục rõ ràng theo từng thành phần hệ thống. Ở mức tổng thể, cấu trúc thư mục chính gồm:

- mobile/: ứng dụng di động dành cho người dùng cuối
- backend/: REST API và xử lý nghiệp vụ
- web-admin/: giao diện quản trị viên
- docs/: tài liệu sơ đồ và mô tả hệ thống

Trong đó, ứng dụng mobile được tổ chức theo hướng feature-based architecture, tức là chia mã nguồn theo từng nhóm chức năng nghiệp vụ thay vì chia theo kiểu toàn bộ model, view, controller riêng lẻ. Bên trong mobile/lib/, các thư mục quan trọng gồm:

- core/: chứa cấu hình, theme, router, API client, i18n, các dịch vụ dùng chung
- features/auth/: xác thực người dùng
- features/home/: trang chủ, khám phá, điều hướng chính
- features/places/: quản lý địa điểm học tập, bản đồ, chỉ đường
- features/document/: quản lý tài liệu học tập
- features/groups/: nhóm học tập, chat nhóm, chia sẻ tài nguyên
- features/friendships/: kết bạn và quản lý lời mời
- features/study_near_me/: gợi ý địa điểm gần vị trí hiện tại
- features/profile/, favorites/, notifications/, settings/, feedback/, report/: hồ sơ cá nhân, yêu thích, thông báo, cài đặt, góp ý, báo cáo
- shared/: các widget dùng chung trong toàn ứng dụng

Trong từng feature, mã nguồn tiếp tục được chia thành các lớp như:

- presentation/: giao diện và controller
- data/models/: mô hình dữ liệu

- data/services/: lớp gọi API hoặc dịch vụ dữ liệu
- data/repositories/: tầng trung gian giữa giao diện và nguồn dữ liệu

Cách tổ chức này giúp dự án có tính mô-đun cao, dễ quản lý, dễ mở rộng thêm chức năng mới, đồng thời hạn chế việc các phần mã nguồn phụ thuộc chặt chẽ vào nhau.

Phía backend cũng được tổ chức tương đối rõ ràng, bao gồm:

- src/routes/: định nghĩa các nhóm API endpoint
- src/services/: xử lý logic nghiệp vụ
- src/dto/: định nghĩa cấu trúc dữ liệu đầu vào/đầu ra
- src/lib/: cấu hình hệ thống, Firebase, Prisma, tiện ích dùng chung
- src/middleware/: JWT middleware, error middleware, RBAC middleware
- docs/: tài liệu schema Firestore và hướng dẫn triển khai

Nhờ cách tổ chức như vậy, hệ thống backend có thể duy trì được tính rõ ràng giữa tầng định tuyến, tầng nghiệp vụ và tầng cấu hình, hỗ trợ tốt cho việc bảo trì và phát triển tiếp theo.

4.1.3 Kiến trúc cài đặt và mô hình hoạt động của hệ thống

Dựa trên mã nguồn thực tế, Sfinity được triển khai theo mô hình client–server kết hợp dịch vụ đám mây.

- Client là ứng dụng Flutter trên thiết bị người dùng.
- Backend server là dịch vụ Express + TypeScript cung cấp các REST API.
- Firebase được dùng cho xác thực, đồng bộ một phần dữ liệu thời gian thực, chat nhóm và lưu trữ tệp.
- SQLite trên thiết bị được dùng để lưu phiên đăng nhập và thiết lập cục bộ.

Trong quá trình hoạt động, khi người dùng thao tác trên giao diện, sự kiện sẽ được chuyển đến các lớp controller ở tầng trình bày. Controller gọi repository tương ứng để xử lý tác vụ. Repository tiếp tục giao tiếp với các service như API service, Firebase

service hoặc local database service. Sau khi dữ liệu được truy xuất hoặc cập nhật, kết quả được ánh xạ thành model và trả ngược lại cho giao diện để hiển thị.

Kiến trúc này có đặc điểm gần với MVVM kết hợp kiến trúc phân lớp:

- Page, Widget đóng vai trò tương tự View
- Các Controller kế thừa ChangeNotifier đóng vai trò gần với ViewModel
- Repository, Service, Model đảm nhiệm tầng dữ liệu và nghiệp vụ

Như vậy, ứng dụng không đi theo mô hình MVC thuần túy mà thiên về hướng tách biệt giao diện, trạng thái và dữ liệu, phù hợp với ứng dụng Flutter quy mô vừa và nhiều tính năng như Sfinity.

4.1.4 Cài đặt ứng dụng mobile

Phần mobile là trung tâm trải nghiệm của người dùng. Từ mã nguồn có thể thấy ứng dụng được khởi tạo bằng cách:

1. Nạp file cấu hình môi trường,
2. Khởi tạo Firebase,
3. Đồng bộ danh sách tiện ích địa điểm,
4. Tạo các repository, service, controller dùng chung,
5. Khởi chạy MaterialApp.router.

Ứng dụng sử dụng `go_router` để điều hướng giữa các màn hình, đồng thời hỗ trợ theme sáng/tối, đa ngôn ngữ và quản lý trạng thái xác thực ở cấp ứng dụng. Các repository quan trọng như tài liệu, địa điểm, yêu thích, nhóm học, bạn bè, đánh giá địa điểm đều được khởi tạo tập trung tại lớp ứng dụng chính trước khi được dùng ở các màn hình tương ứng.

Một điểm nổi bật là phần xác thực không chỉ làm việc với backend mà còn kết hợp với Firebase Auth và Firestore. Khi người dùng đăng nhập bằng email/mật khẩu, hệ thống trước hết xác thực với backend để nhận JWT token, đồng thời đồng bộ phiên người dùng với Firebase. Khi đăng nhập bằng Google hoặc Facebook, ứng dụng sử dụng Firebase để xác thực mạng xã hội, sau đó gửi Firebase ID token về backend để

đổi lấy access token của hệ thống. Cơ chế này giúp kết hợp được ưu điểm của đăng nhập xã hội và quản lý phiên tập trung của backend.

Ngoài ra, phiên đăng nhập và một số dữ liệu cá nhân cơ bản như tên, email, avatar, ngày sinh, giới tính, địa chỉ được lưu cục bộ bằng SQLite. Điều này giúp ứng dụng có thể khôi phục trạng thái nhanh và cải thiện trải nghiệm khi mở lại ứng dụng.

4.1.5 Cài đặt backend và các module API

Backend Sfinity cung cấp tập API tương đối đầy đủ cho các nghiệp vụ chính của hệ thống. Qua mã nguồn định tuyến, các nhóm endpoint chính bao gồm:

- /auth: đăng ký, đăng nhập, hồ sơ, OTP, đăng nhập Firebase
- /users: quản lý thông tin người dùng
- /document: tài liệu học tập
- /places: địa điểm học tập, check-in, review, ảnh
- /study-near-me: gợi ý địa điểm gần vị trí
- /explore: nội dung nổi bật, thống kê, top người dùng
- /favorites: danh sách yêu thích
- /friends: kết bạn và lời mời kết bạn
- /groups: nhóm học tập và thành viên
- /notifications: thông báo
- /feedback, /reports: góp ý và báo cáo
- /amenities, /settings, /upload: tiện ích, cấu hình và tải tệp
- /admin/dashboard: thống kê quản trị

Phía backend sử dụng Express để tiếp nhận request, sau đó chuyển tiếp đến từng router tương ứng. Mỗi router tiếp tục gọi service để xử lý nghiệp vụ. Dữ liệu được kiểm tra thông qua DTO và middleware, từ đó đảm bảo tính hợp lệ trước khi ghi vào hệ thống.

Một đặc điểm quan trọng là Firestore là nơi lưu trữ dữ liệu nghiệp vụ chính như người dùng, tài liệu, địa điểm, nhóm, đánh giá, bạn bè, báo cáo, thông báo. Trong khi đó, Prisma với SQLite được dùng cho một số thành phần hỗ trợ và metadata. Việc sử dụng Firestore giúp hệ thống linh hoạt hơn trong lưu trữ NoSQL, đặc biệt phù hợp với các dữ liệu như nhóm, quan hệ xã hội, thông báo và chat.

Backend cũng tích hợp Swagger UI để hỗ trợ kiểm thử và mô tả tài liệu API. Đây là yếu tố thuận lợi trong quá trình phát triển vì giúp nhóm dễ dàng kiểm tra endpoint trước khi tích hợp vào mobile.

4.1.6 Cài đặt cơ chế xử lý dữ liệu

Cơ chế xử lý dữ liệu của Sfinity không chỉ dựa trên một nguồn duy nhất mà là sự kết hợp giữa nhiều thành phần.

- Dữ liệu qua REST API: Phần lớn nghiệp vụ như đăng nhập, quản lý tài liệu, địa điểm, đánh giá, bạn bè, nhóm học, thống kê, báo cáo đều đi qua backend REST API. Ứng dụng mobile sử dụng Dio thông qua lớp ApiClient để gửi request đến backend, kèm theo JWT token trong header Authorization khi người dùng đã đăng nhập.
- Dữ liệu qua Firebase Firestore: Một số chức năng thời gian thực, đặc biệt là chat nhóm, sử dụng Firestore trực tiếp từ mobile. Tin nhắn nhóm được lưu trong collection `groups/{groupId}/messages`, từ đó ứng dụng có thể stream realtime và cập nhật giao diện ngay khi có tin nhắn mới.
- Tập tin qua Firebase Storage: Khi người dùng gửi ảnh hoặc file vào nhóm, ứng dụng upload trực tiếp tập lên Firebase Storage, nhận URL tải về rồi lưu URL này trong bản ghi tin nhắn Firestore. Nhờ đó, hệ thống hỗ trợ chia sẻ ảnh, tài liệu và tài nguyên trong nhóm một cách thuận tiện.
- Dữ liệu cục bộ qua SQLite: SQLite được sử dụng để lưu phiên đăng nhập, access token, thông tin người dùng cơ bản và một số app settings.

Việc lưu cục bộ như vậy giúp ứng dụng giảm số lần gọi mạng không cần thiết, đồng thời cải thiện khả năng duy trì trạng thái đăng nhập.

Từ đó có thể thấy luồng dữ liệu trong Sfinity được tổ chức theo hướng lai ghép:

- Backend API xử lý nghiệp vụ trung tâm
- Firestore/Storage hỗ trợ thời gian thực và lưu tệp
- SQLite hỗ trợ lưu trữ cục bộ

Đây là một giải pháp phù hợp với các ứng dụng di động có yếu tố mạng xã hội và học tập cộng đồng.

4.1.7 Các module chức năng chính đã thực hiện

Dựa trên mã nguồn hiện có, hệ thống đã hiện thực tương đối đầy đủ các nhóm chức năng sau:

- Xác thực người dùng
 - Đăng ký tài khoản
 - Đăng nhập bằng email/mật khẩu
 - Đăng nhập Google
 - Quên mật khẩu, OTP
 - Cập nhật hồ sơ
 - Lưu phiên đăng nhập
- Khám phá và trang chủ
 - Tìm kiếm nội dung
 - Nội dung nổi bật
 - Hoạt động tuần
 - Top người dùng đóng góp
- Địa điểm học tập
 - Hiển thị bản đồ OpenStreetMap
 - Tìm kiếm và lọc địa điểm
 - Xem chi tiết địa điểm
 - Check-in

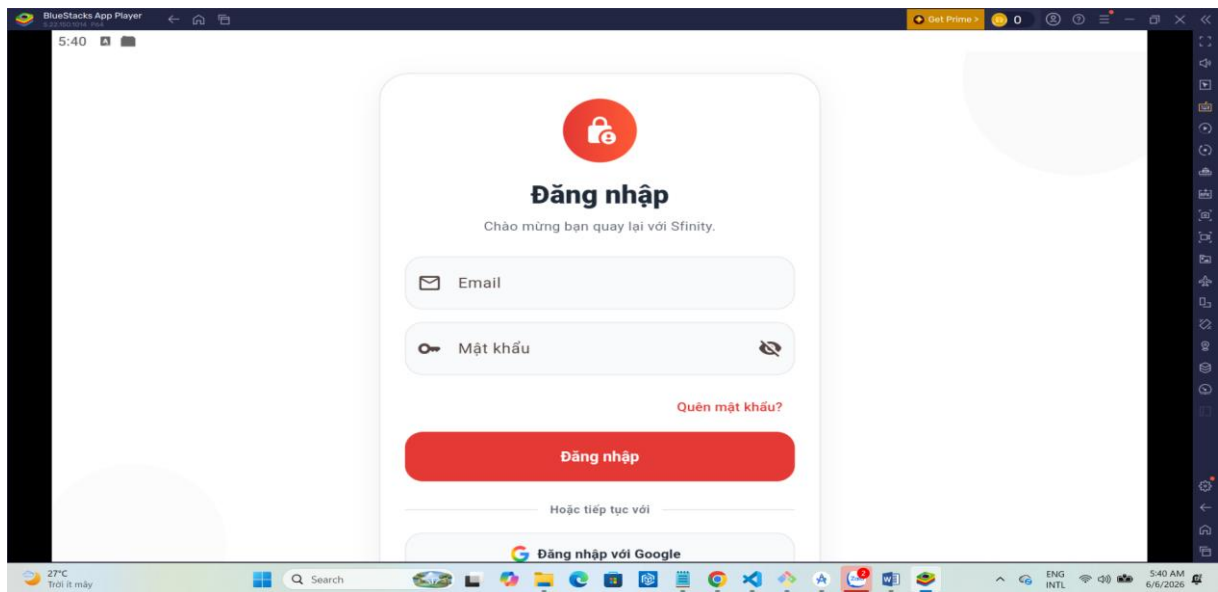
- Đánh giá địa điểm
- Thêm/sửa địa điểm cá nhân
- Chỉ đường
- Học gần tôi
- Tài liệu học tập
 - Xem danh sách tài liệu
 - Xem chi tiết tài liệu
 - Tải xuống
 - Tạo/sửa tài liệu
 - Liên kết tài liệu với địa điểm
 - Quản lý tài liệu cá nhân
- Cộng đồng học tập
 - Kết bạn, lời mời kết bạn
 - Tạo nhóm học tập
 - Thêm thành viên
 - Chat nhóm thời gian thực
 - Chia sẻ file, ảnh, tài liệu, vị trí trong nhóm
 - Lưu trữ tài nguyên nhóm
 - Bản đồ vị trí thành viên
- Cá nhân và hỗ trợ
 - Hồ sơ người dùng
 - Yêu thích
 - Thông báo
 - Phản hồi
 - Báo cáo vi phạm

- Cài đặt ngôn ngữ và giao diện

4.1.8 Gợi ý minh họa hình ảnh giao diện trong báo cáo

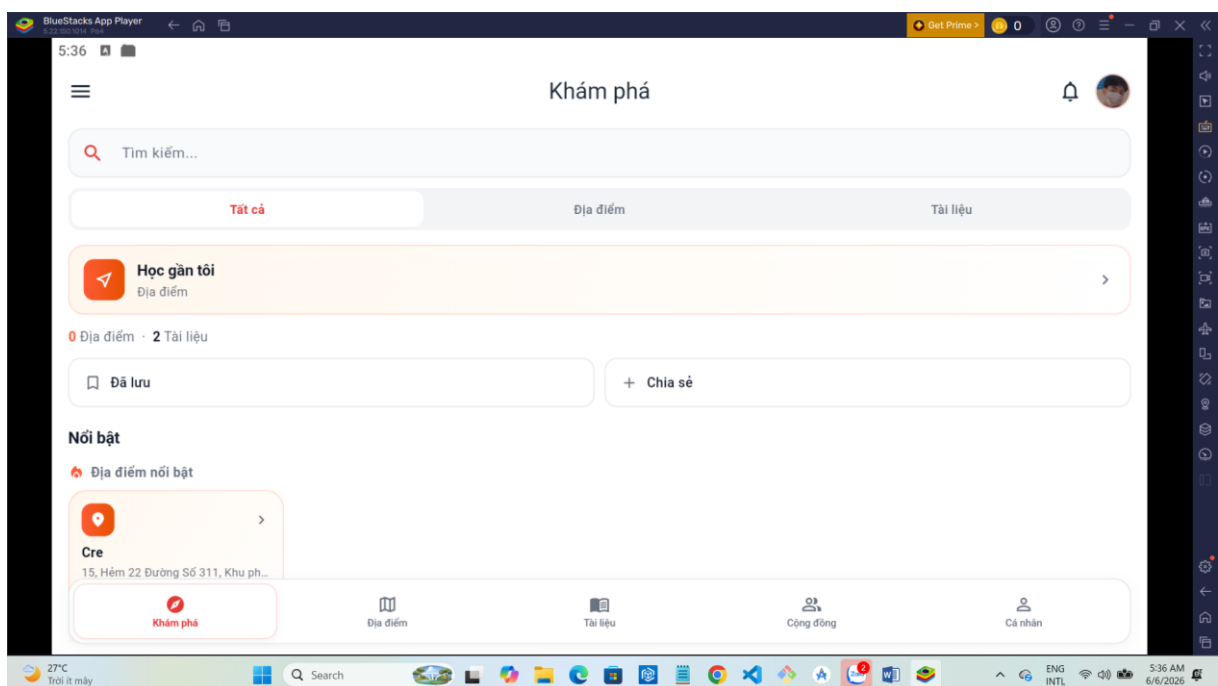
Để phân cài đặt hệ thống trực quan hơn, trong báo cáo nên bổ sung ảnh chụp từ ứng dụng thực tế, cụ thể là ứng dụng BlueStacks tại các nhóm màn hình sau:

- Màn hình đăng nhập / đăng ký



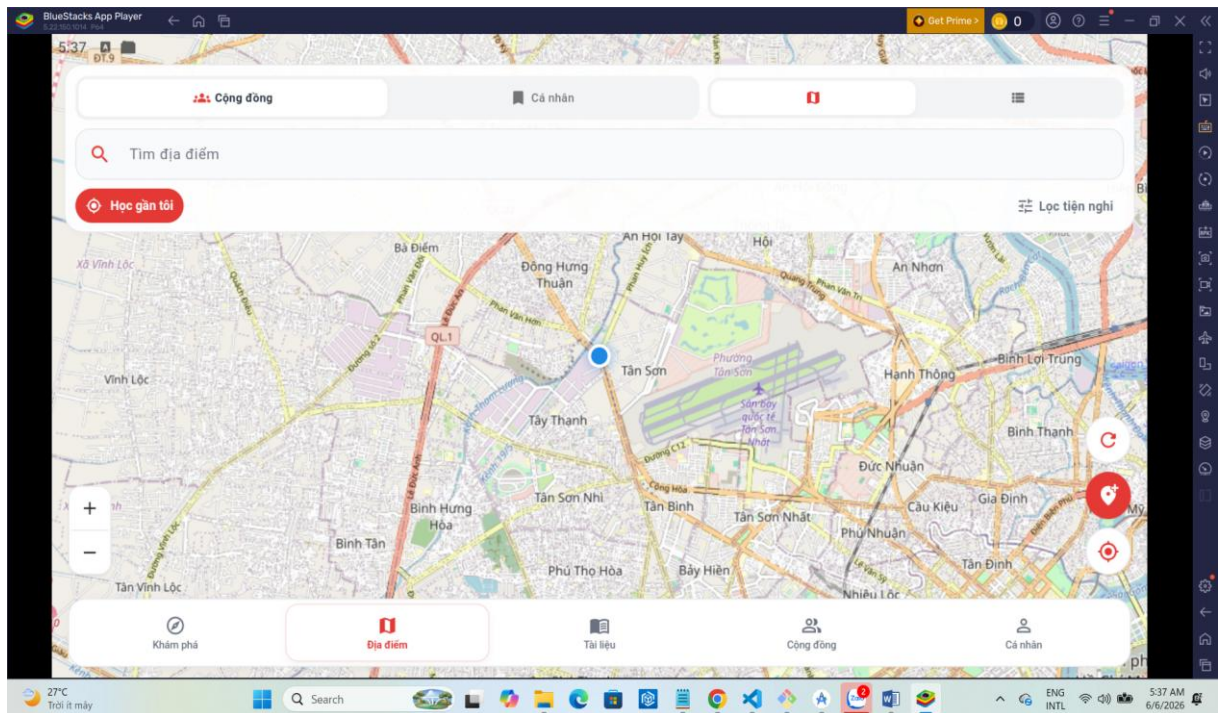
Hình 4.1 Giao diện đăng nhập / đăng ký trên ứng dụng thực tế

- Màn hình trang chủ và khám phá



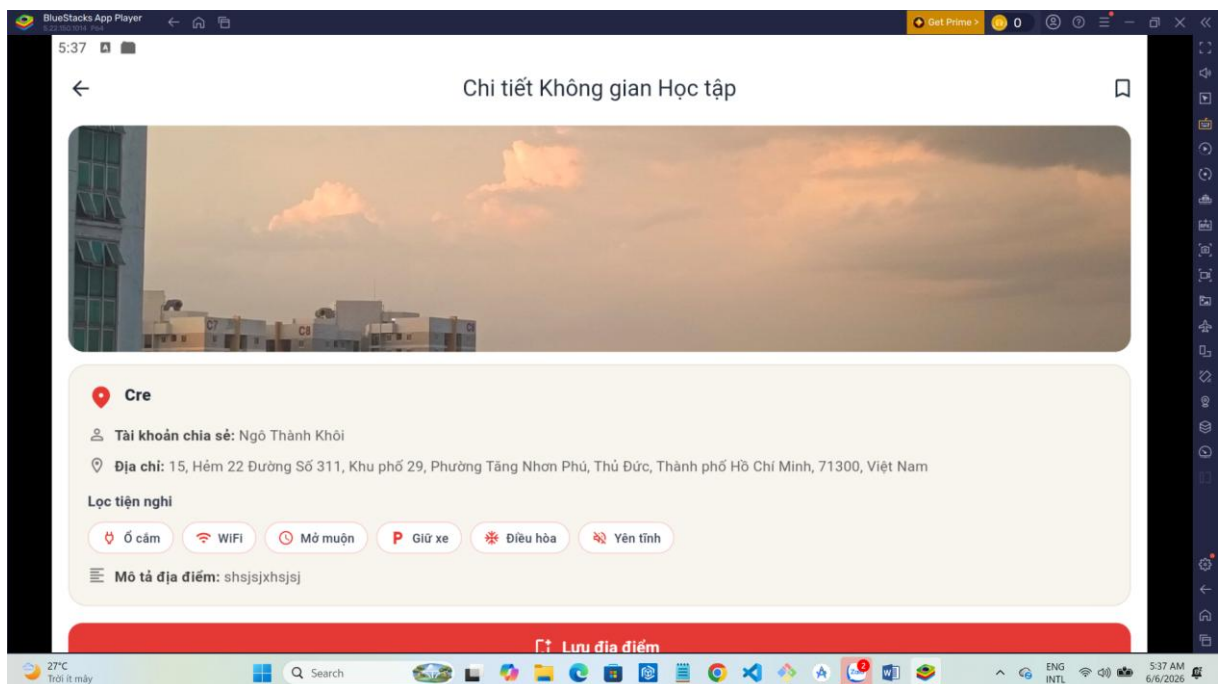
Hình 4.2 Giao diện trang chủ và khám phá trên ứng dụng thực tế

- Màn hình tìm kiếm địa điểm trên bản đồ



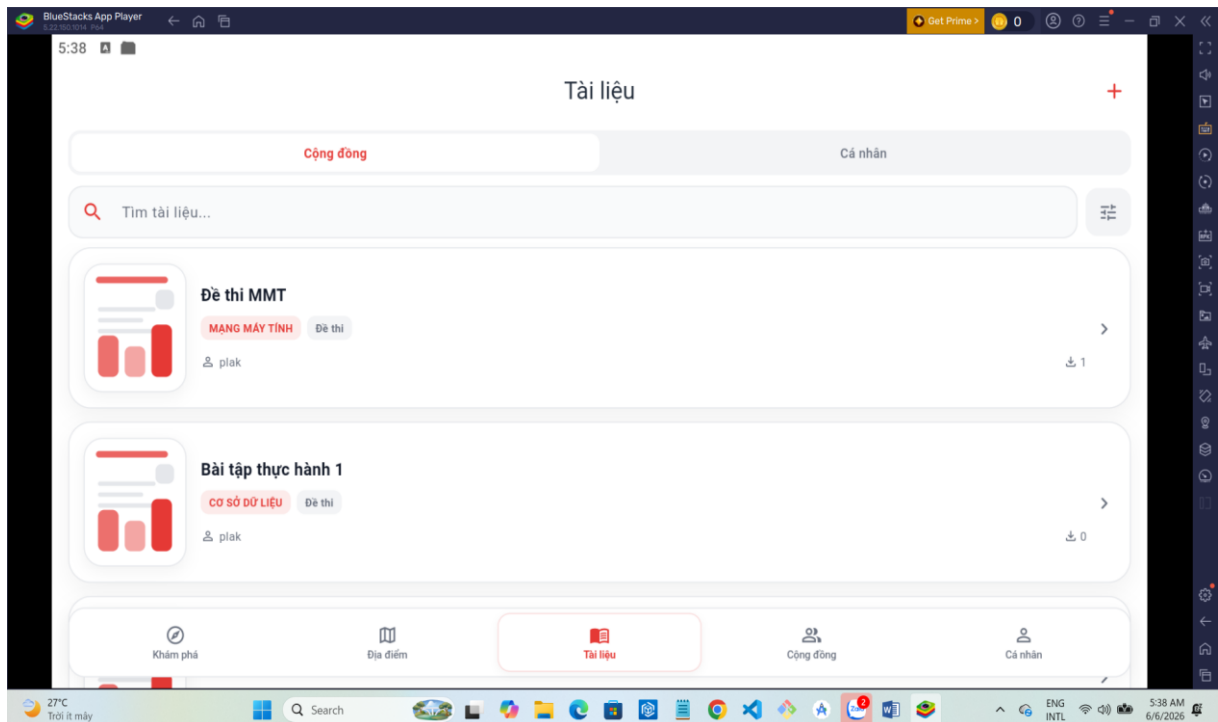
Hình 4.3 Giao diện tìm kiếm địa điểm trên ứng dụng thực tế

- Màn hình chi tiết địa điểm



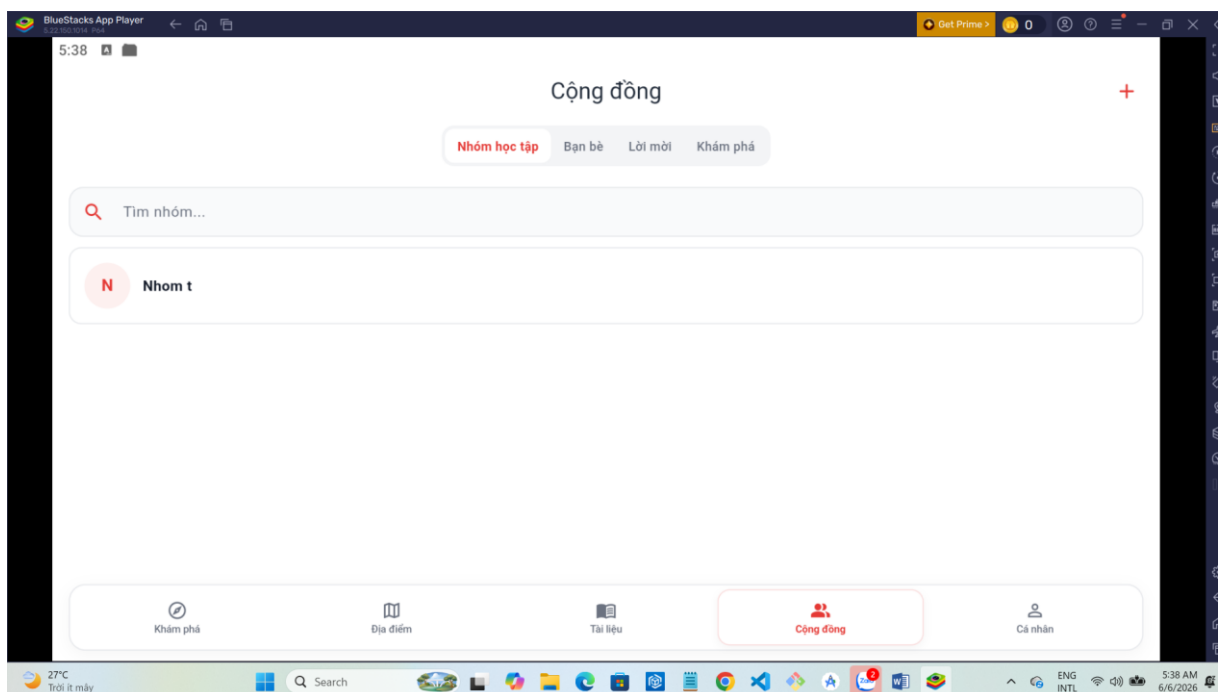
Hình 4.4 Giao diện chi tiết địa điểm trên ứng dụng thực tế

- Màn hình danh sách tài liệu



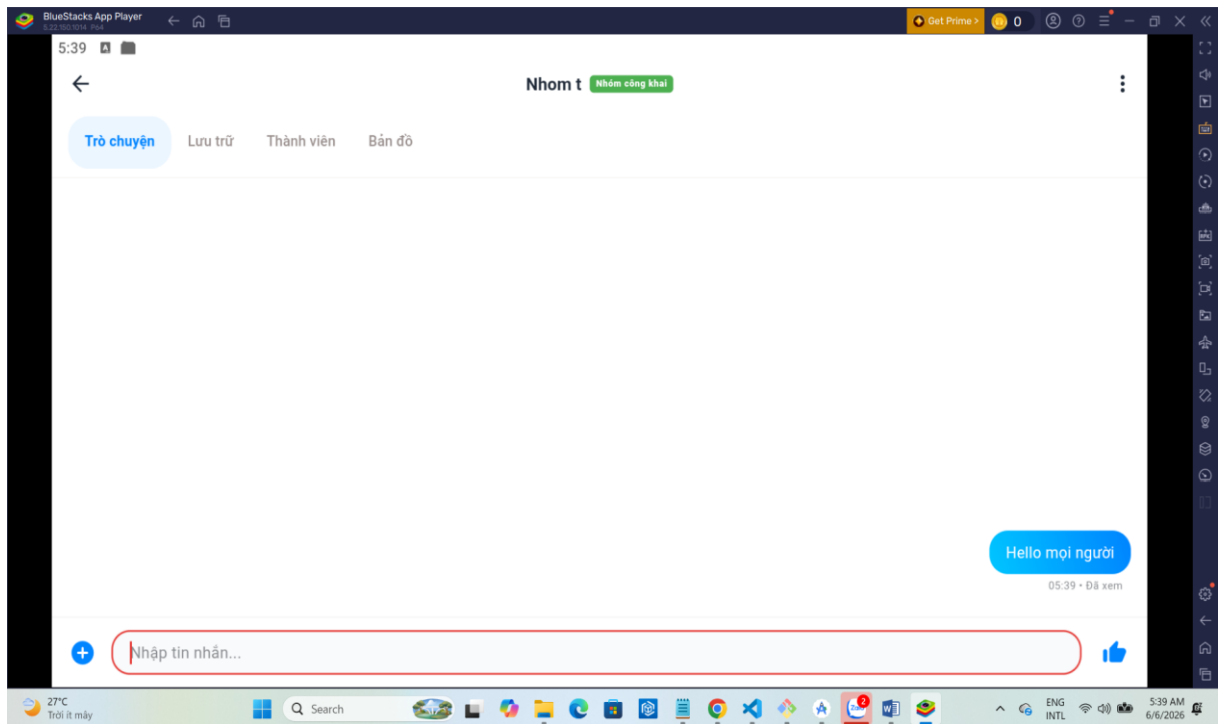
Hình 4.5 Giao diện danh sách tài liệu trên ứng dụng thực tế

- Màn hình tạo nhóm học tập



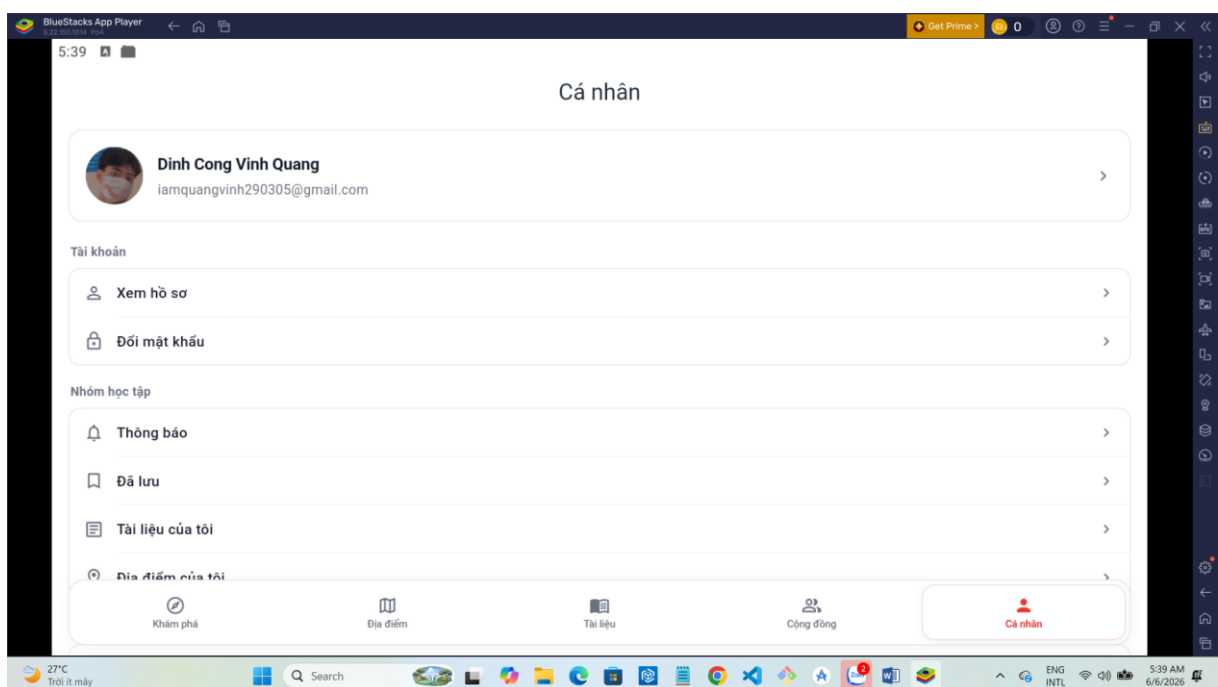
Hình 4.6 Giao diện nhóm học tập trên ứng dụng thực tế

- Màn hình chat nhóm



Hình 4.7 Giao diện chat nhóm trên ứng dụng thực tế

- Màn hình hồ sơ cá nhân và cài đặt



Hình 4.8 Giao diện hồ sơ cá nhân và cài đặt

4.2 Mô tả quá trình sử dụng công cụ AI hỗ trợ phát triển hệ thống

Trong quá trình xây dựng hệ thống Sfinity, nhóm không chỉ sử dụng các công cụ lập trình truyền thống mà còn kết hợp các nền tảng trí tuệ nhân tạo hỗ trợ lập trình nhằm

tăng tốc độ phát triển, hỗ trợ học công nghệ mới và cải thiện chất lượng sản phẩm. Việc ứng dụng AI đặc biệt phù hợp trong bối cảnh dự án tích hợp nhiều công nghệ cùng lúc như Flutter, Firebase, Firestore, REST API, SQLite và bản đồ số.

4.2.1 Lý do lựa chọn công cụ AI

Nhóm sử dụng AI Coding Assistant/AI Agent vì một số lý do chính:

- Hỗ trợ gợi ý nhanh cấu trúc mã nguồn ban đầu,
- Rút ngắn thời gian tra cứu tài liệu công nghệ,
- Hỗ trợ giải thích lỗi và đề xuất hướng sửa,
- Hỗ trợ sinh mã giao diện, model, service và quy trình xử lý thường gặp,
- Hỗ trợ tối ưu hoặc chuẩn hóa một số đoạn mã lặp lại.

Các công cụ AI được sử dụng có thể bao gồm Cursor AI, Claude, GitHub Copilot/Codex hoặc các nền tảng tương đương. Trong đó, AI được dùng như công cụ hỗ trợ kỹ thuật, không thay thế vai trò ra quyết định của nhóm phát triển.

4.2.2 AI hỗ trợ trong từng giai đoạn phát triển

a) Giai đoạn phân tích và thiết kế

Ở giai đoạn đầu, AI được dùng để:

- Gợi ý phân rã chức năng hệ thống,
- Đề xuất cách chia module trong project Flutter,
- Gợi ý kiến trúc phù hợp như feature-based, MVVM hoặc phân lớp,
- Hỗ trợ phác thảo sơ đồ dữ liệu và quan hệ thực thể.

Nhờ đó, nhóm có thể rút ngắn thời gian định hình cấu trúc ban đầu của hệ thống.

b) Giai đoạn hiện thực giao diện và luồng người dùng

AI được dùng để hỗ trợ:

- Sinh khung giao diện cho các màn hình như đăng nhập, trang chủ, danh sách tài liệu, màn hình chat,
- Đề xuất cách xây dựng widget tái sử dụng,

- Hỗ trợ xử lý điều hướng giữa các màn hình,
- Gợi ý cách tổ chức state cho controller và form nhập liệu.

Tuy nhiên, giao diện sinh bởi AI thường chỉ là bản khởi đầu. Nhóm vẫn phải tự điều chỉnh bố cục, màu sắc, trải nghiệm thao tác và tính nhất quán với toàn bộ ứng dụng.

c) Giai đoạn xây dựng tầng dữ liệu và API

Ở phần backend và tích hợp dữ liệu, AI được dùng để:

- Gợi ý cấu trúc endpoint REST API,
- Sinh DTO và service cơ bản,
- Hỗ trợ viết các hàm CRUD,
- Gợi ý cách kết nối Flutter với API thông qua client/repository,
- Hỗ trợ tích hợp Firebase Auth, Firestore, Firebase Storage và SQLite.

Đặc biệt, khi dự án cần phối hợp cả backend và Firebase, AI giúp nhóm nhanh chóng xây dựng các đoạn mã mẫu để thử nghiệm luồng xử lý.

d) Giai đoạn sửa lỗi và tinh chỉnh

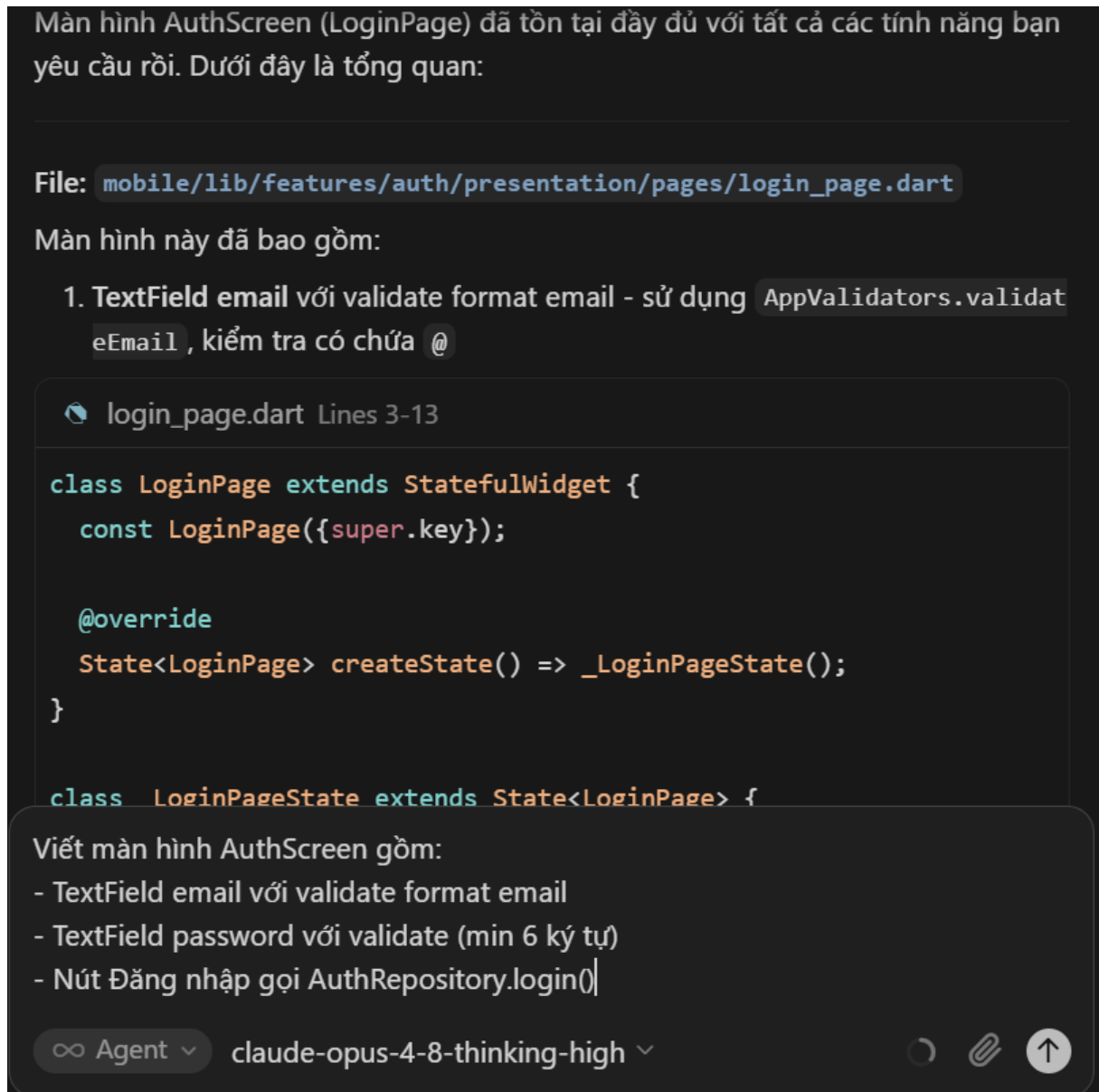
Trong quá trình phát triển, AI còn được sử dụng để:

- Giải thích log lỗi,
- Tìm nguyên nhân sai khác dữ liệu giữa mobile và backend,
- Gợi ý sửa lỗi Firebase Auth, lỗi gọi API, lỗi parse dữ liệu,
- Hỗ trợ rà soát các đoạn mã xử lý bất đồng bộ,
- Đề xuất cách tái cấu trúc đoạn mã chưa rõ ràng.

4.2.3 Cách nhóm sử dụng prompt và kiểm soát chất lượng

Khi sử dụng AI, nhóm không chỉ nhập yêu cầu chung chung mà thường mô tả bài toán theo ngữ cảnh cụ thể, ví dụ:

- Yêu cầu xây dựng màn hình đăng nhập Flutter có validate email và password
- Yêu cầu viết repository gọi API lấy danh sách địa điểm,
- Yêu cầu xử lý lỗi khi xác thực Firebase và backend không đồng bộ,
- Yêu cầu đề xuất cách lưu session bằng SQLite.



Hình 4.9 Ví dụ yêu cầu sử dụng AI

Sau khi AI sinh mã, nhóm luôn thực hiện các bước kiểm tra lại:

1. Đọc và hiểu logic của đoạn mã được đề xuất;

2. Đối chiếu với yêu cầu nghiệp vụ ban đầu;
3. Chạy thử nghiệm thực tế;
4. Chính sửa lại tên biến, cấu trúc hàm, dữ liệu trả về;
5. Tích hợp vào kiến trúc chung của project;
6. Kiểm tra lại hiệu năng và tính ổn định.

Điều này rất quan trọng vì mã do AI sinh ra có thể đúng cú pháp nhưng chưa chắc phù hợp với hệ thống thực tế hoặc có thể tồn tại lỗi logic tiềm ẩn.

4.2.4 Vai trò thực tế của người phát triển

Trong toàn bộ quá trình phát triển, AI chỉ đóng vai trò công cụ hỗ trợ, còn vai trò trung tâm vẫn thuộc về nhóm thực hiện đề tài. Người phát triển là người:

- Phân tích yêu cầu nghiệp vụ,
- Quyết định kiến trúc hệ thống,
- Lựa chọn công nghệ,
- Đánh giá tính đúng đắn của mã nguồn,
- Phát hiện lỗi logic,
- Tích hợp các thành phần thành sản phẩm hoàn chỉnh.

Do đó, việc sử dụng AI không làm giảm vai trò của người lập trình, mà ngược lại đòi hỏi người sử dụng phải có tư duy kỹ thuật đủ tốt để kiểm soát chất lượng đầu ra.

4.2.5 Đánh giá hiệu quả của AI trong dự án

Việc ứng dụng AI vào phát triển Sfinity mang lại nhiều lợi ích:

- Tiết kiệm thời gian viết mã khung,
- Tăng tốc độ thử nghiệm ý tưởng,
- Hỗ trợ học nhanh thư viện và công nghệ mới,
- Giảm thời gian tra cứu lỗi cơ bản,
- Cải thiện tốc độ phát triển tính năng.

Tuy nhiên, AI cũng có một số hạn chế:

- Đôi khi sinh mã chưa tối ưu,
- Có thể đề xuất giải pháp không hoàn toàn phù hợp kiến trúc project,
- Thông tin hoặc API gợi ý có lúc chưa chính xác,
- Dễ tạo cảm giác phụ thuộc nếu người dùng không hiểu bản chất hệ thống.

Vì vậy, nhóm chỉ sử dụng AI như công cụ đồng hành, không phụ thuộc hoàn toàn vào AI trong các quyết định quan trọng.

4.3 Thực nghiệm hệ thống

Phần thực nghiệm được tiến hành nhằm kiểm tra khả năng hoạt động của hệ thống trong các tình huống sử dụng điển hình. Việc thực nghiệm tập trung vào cả kiểm thử chức năng, kiểm thử giao diện, kiểm thử khả năng xử lý dữ liệu và kiểm thử mức độ ổn định khi phối hợp nhiều thành phần như mobile, backend, Firebase và SQLite.

4.3.1 Mục tiêu thực nghiệm

Các mục tiêu chính của thực nghiệm bao gồm:

- Kiểm tra tính đúng đắn của các chức năng cốt lõi;
- Đánh giá khả năng tương tác giữa mobile và backend;
- Đánh giá khả năng đồng bộ dữ liệu qua Firebase;
- Kiểm tra tính ổn định của ứng dụng trên thiết bị di động;
- Đánh giá trải nghiệm người dùng cơ bản.

4.3.2 Môi trường thực nghiệm

Hệ thống được thực nghiệm trong môi trường phát triển gồm:

- Backend chạy cục bộ tại `http://localhost:3000/api`,
- Mobile chạy trên thiết bị/emulator Flutter,
- Firebase được cấu hình cho xác thực, Firestore và Storage,
- SQLite hoạt động cục bộ trên thiết bị để lưu session và setting.

Việc kiểm thử được thực hiện với dữ liệu mẫu và tài khoản thử nghiệm, bao gồm tài khoản người dùng thông thường và tài khoản quản trị.

4.3.3 Các kịch bản thử nghiệm chức năng

a) Thử nghiệm chức năng xác thực

- Đăng ký tài khoản mới bằng email/mật khẩu;
- Xác thực email và đăng nhập;
- Đăng nhập bằng Google;
- Gửi yêu cầu quên mật khẩu;
- Cập nhật hồ sơ người dùng;
- Kiểm tra khả năng lưu và khôi phục phiên đăng nhập.

Kết quả mong đợi: hệ thống tạo được tài khoản hợp lệ, lưu được token, hiển thị đúng thông tin người dùng và khôi phục được phiên khi mở lại ứng dụng.

b) Thử nghiệm chức năng địa điểm học tập

- Tải danh sách địa điểm;
- Hiển thị địa điểm trên bản đồ;
- Lọc và tìm kiếm địa điểm;
- Xem chi tiết địa điểm;
- Check-in tại địa điểm;
- Gửi đánh giá và ảnh địa điểm;
- Thử chức năng học gần tôi.

Kết quả mong đợi: dữ liệu địa điểm được tải đầy đủ, phản hồi đúng theo vị trí hiện tại và các thao tác check-in, đánh giá được lưu thành công.

c) Thử nghiệm chức năng tài liệu học tập

- Xem danh sách tài liệu;
- Mở chi tiết tài liệu;

- Tải hoặc chia sẻ tài liệu;
- Tạo mới tài liệu;
- Sửa thông tin tài liệu;
- Liên kết tài liệu với địa điểm.

Kết quả mong đợi: tài liệu được quản lý đúng, hiển thị đúng thông tin chi tiết và thao tác tạo/sửa/tải xuống hoạt động ổn định.

d) Thực nghiệm chức năng nhóm học tập và cộng đồng

- Tạo nhóm học tập mới;
- Mời hoặc thêm thành viên;
- Gửi tin nhắn văn bản;
- Chia sẻ file, ảnh, tài liệu và vị trí;
- Xem tài nguyên chung trong nhóm;
- Kiểm tra bản đồ vị trí thành viên;
- Thực hiện kết bạn và xử lý lời mời.

Kết quả mong đợi: dữ liệu nhóm được tạo chính xác, tin nhắn được cập nhật realtime, tài nguyên chia sẻ hiển thị đúng và quan hệ bạn bè được quản lý nhất quán.

e) Thực nghiệm các chức năng hỗ trợ

- Lưu danh sách yêu thích;
- Nhận và xem thông báo;
- Gửi phản hồi;
- Gửi báo cáo nội dung;
- Thay đổi theme và ngôn ngữ.

Kết quả mong đợi: các thiết lập được lưu giữ, dữ liệu phản hồi và báo cáo được gửi thành công, giao diện phản ánh đúng cấu hình đã chọn.

4.3.4 Kiểm thử giao diện và trải nghiệm sử dụng

Ngoài kiểm thử chức năng, giao diện cũng được đánh giá theo các tiêu chí:

- Bố cục dễ nhìn, dễ thao tác;
- Điều hướng rõ ràng giữa các nhóm chức năng;
- Hiện thị hợp lý trên các kích thước màn hình khác nhau;
- Trạng thái loading, empty state, error state được xử lý rõ ràng;
- Màu sắc và typography nhất quán.

Kết quả cho thấy giao diện ứng dụng tương đối đồng bộ, hiện đại và phù hợp với người dùng sinh viên. Các module như bản đồ, tài liệu, nhóm học và hồ sơ cá nhân có mức độ trực quan tốt, hỗ trợ người dùng thao tác thuận tiện.

4.3.5 Kiểm thử hiệu năng và ổn định

Một số nội dung được kiểm thử thêm gồm:

- Thời gian phản hồi khi đăng nhập;
- Thời gian tải danh sách địa điểm và tài liệu;
- Tốc độ cập nhật tin nhắn nhóm theo thời gian thực;
- Khả năng xử lý khi mạng chậm hoặc gián đoạn;
- Khả năng giữ trạng thái đăng nhập khi khởi động lại ứng dụng.

Kết quả thực nghiệm cho thấy hệ thống hoạt động ổn định ở các thao tác chính. Những tính năng sử dụng Firestore như chat nhóm có khả năng cập nhật nhanh. Các chức năng gọi REST API có tốc độ phản hồi chấp nhận được trong môi trường thử nghiệm. Tuy nhiên, trải nghiệm vẫn phụ thuộc vào chất lượng kết nối mạng, đặc biệt ở các thao tác tải file, đồng bộ dữ liệu hoặc truy vấn vị trí.

4.4 Đánh giá hệ thống

4.4.1 Mức độ hoàn thiện chức năng

Xét về chức năng, hệ thống đã triển khai được tương đối đầy đủ các thành phần quan trọng:

- Xác thực người dùng,

- Quản lý hồ sơ,
- Tìm kiếm và khám phá nội dung,
- Quản lý địa điểm học tập,
- Quản lý tài liệu,
- Nhóm học tập và trò chuyện,
- Kết bạn và tương tác cộng đồng,
- Phản hồi, báo cáo, thông báo,
- Cấu hình theme và ngôn ngữ.

Đây là một mức độ hoàn thiện tương đối cao đối với một ứng dụng di động học tập cộng đồng.

4.4.2 Đánh giá về kiến trúc và tổ chức hệ thống

Kiến trúc hệ thống được tổ chức hợp lý, kết hợp giữa backend API, Firebase và SQLite. Việc chia source code theo từng feature giúp dễ bảo trì, còn việc phân tách giữa giao diện, controller, repository và service làm cho luồng xử lý rõ ràng hơn. Đây là ưu điểm lớn giúp hệ thống có khả năng mở rộng về sau.

4.4.3 Đánh giá về hiệu năng và tính ổn định

Hệ thống có khả năng phản hồi tốt trong các thao tác thông thường như đăng nhập, tải dữ liệu, tìm kiếm và cập nhật nội dung. Chat nhóm và chia sẻ tài nguyên hoạt động linh hoạt nhờ Firestore và Firebase Storage. Việc sử dụng SQLite giúp duy trì session và cải thiện tốc độ mở lại ứng dụng.

Tuy nhiên, với các thao tác phụ thuộc mạnh vào mạng như tải tệp, lấy dữ liệu bản đồ hoặc đồng bộ thông báo, hiệu năng vẫn còn phụ thuộc vào chất lượng kết nối Internet. Đây là hạn chế tự nhiên của các ứng dụng có tích hợp nhiều dịch vụ mạng.

4.4.4 Đánh giá về trải nghiệm người dùng

Ứng dụng có giao diện hiện đại, các nhóm chức năng được phân chia rõ ràng, phù hợp với đối tượng sinh viên. Người dùng có thể dễ dàng truy cập các chức năng chính

như xem địa điểm, chia sẻ tài liệu, tham gia nhóm học, trò chuyện và cập nhật hồ sơ. Việc hỗ trợ đa ngôn ngữ và theme cũng góp phần tăng tính linh hoạt trong sử dụng.

4.4.5 Ưu điểm của hệ thống

Một số ưu điểm nổi bật của hệ thống gồm:

- Tích hợp nhiều chức năng trong cùng một nền tảng học tập cộng đồng;
- Kết hợp tốt giữa dữ liệu bản đồ, tài liệu và tương tác nhóm;
- Kiến trúc tổ chức rõ ràng, dễ phát triển mở rộng;
- Hỗ trợ tốt các công nghệ hiện đại như Firebase, Flutter và REST API;
- Có khả năng đồng bộ thời gian thực ở các chức năng cần thiết.

4.4.6 Hạn chế của hệ thống

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế:

- Một số chức năng phụ vẫn ở mức placeholder hoặc chưa hoàn thiện đầy đủ;
- Hiệu năng có thể bị ảnh hưởng khi mạng yếu;
- Một số quy trình cần tiếp tục tối ưu về trải nghiệm người dùng;
- Phạm vi kiểm thử thực tế có thể chưa bao phủ hết mọi thiết bị và mọi tình huống biên;
- Hệ thống chưa triển khai đầy đủ một số tính năng nâng cao như 2FA hoặc quản lý phiên chuyên sâu.

4.4.7 Nguyên nhân của hạn chế

Các hạn chế trên chủ yếu xuất phát từ:

- Giới hạn thời gian thực hiện đề tài;
- Nguồn lực phát triển có hạn;
- Khối lượng chức năng tương đối lớn;
- Việc phải tích hợp nhiều công nghệ khác nhau trong cùng một hệ thống;

- Điều kiện kiểm thử thực tế chưa thật sự phong phú.

4.5 Khó khăn và hạn chế trong quá trình triển khai

Trong quá trình triển khai đề tài, nhóm gặp nhiều khó khăn kỹ thuật, đặc biệt ở việc tích hợp nhiều công nghệ cùng lúc như Flutter, Firebase Auth, Firestore, Firebase Storage, REST API và SQLite. Việc đảm bảo dữ liệu nhất quán giữa backend và Firebase là một vấn đề đòi hỏi phải thử nghiệm, sửa lỗi và đồng bộ nhiều lần.

Ngoài ra, các chức năng như bản đồ, định vị, chat realtime, chia sẻ file trong nhóm và đồng bộ session cục bộ đều có độ phức tạp cao hơn so với các ứng dụng CRUD thông thường. Điều này làm tăng khối lượng công việc trong cả giai đoạn phát triển lẫn kiểm thử.

Bên cạnh yếu tố kỹ thuật, nhóm còn gặp hạn chế về thời gian, số lượng thiết bị kiểm thử, điều kiện mạng và phạm vi triển khai thực tế. Do đó, một số chức năng tuy đã có bản hoạt động nhưng vẫn cần thêm thời gian để tối ưu và hoàn thiện hơn.

Kết luận chương: Trong chương này, đề tài đã trình bày quá trình cài đặt và hiện thực hệ thống Sfinity dựa trên cấu trúc project và mã nguồn thực tế, bao gồm tổ chức source code, các module chức năng, kiến trúc xử lý dữ liệu và cơ chế phối hợp giữa Flutter, backend, Firebase và SQLite. Chương cũng đã mô tả việc ứng dụng công cụ AI trong quá trình phát triển, đồng thời tiến hành thực nghiệm và đánh giá hệ thống trên các tiêu chí về chức năng, hiệu năng, tính ổn định và trải nghiệm người dùng.

CHƯƠNG 5 KẾT LUẬN

5.1 Kết quả đạt được

5.1.1 Về mặt lý thuyết

Sau quá trình thực hiện đề tài, nhóm đã củng cố và vận dụng được nhiều kiến thức cơ sở lý thuyết đã trình bày ở chương 2 vào quá trình xây dựng hệ thống thực tế. Trước hết, nhóm hiểu rõ hơn về quy trình phát triển một hệ thống phần mềm hoàn chỉnh, từ giai đoạn khảo sát yêu cầu, phân tích bài toán, thiết kế chức năng, thiết kế dữ liệu cho đến triển khai và kiểm thử hệ thống.

Bên cạnh đó, nhóm đã nắm vững hơn về kiến trúc client-server trong xây dựng ứng dụng hiện đại. Thông qua đề tài, nhóm hiểu được cách một ứng dụng mobile giao tiếp với backend thông qua REST API, cách phân tách giữa giao diện, xử lý nghiệp vụ và tầng dữ liệu, cũng như vai trò của các thành phần như repository, service và controller trong việc tổ chức source code.

Về cơ sở dữ liệu, nhóm đã hiểu sâu hơn về cách xây dựng và quản lý dữ liệu trong môi trường kết hợp nhiều công nghệ, đặc biệt là Firebase Firestore, Firebase Storage và SQLite. Nhóm cũng có thêm kiến thức về mô hình lưu trữ dữ liệu NoSQL, cách tổ chức collection/document, cách lưu trữ dữ liệu cục bộ trên thiết bị và cách đồng bộ dữ liệu giữa nhiều nguồn khác nhau.

Ngoài ra, nhóm đã mở rộng kiến thức về phát triển ứng dụng di động bằng Flutter, xử lý định tuyến giao diện, quản lý trạng thái, làm việc với bản đồ số, định vị người dùng và triển khai các chức năng tương tác cộng đồng như chat nhóm, kết bạn và chia sẻ tài nguyên. Không chỉ dừng lại ở kiến thức lập trình, nhóm còn hiểu thêm về cách sử dụng công cụ AI hỗ trợ phát triển phần mềm, biết cách khai thác AI để gợi ý giải pháp, sinh mã nguồn, hỗ trợ sửa lỗi và tối ưu quy trình làm việc.

Nhìn chung, về mặt lý thuyết, đề tài không chỉ giúp nhóm củng cố những kiến thức đã học mà còn giúp tiếp cận thêm nhiều công nghệ và phương pháp phát triển phần mềm hiện đại, từ đó nâng cao tư duy phân tích, thiết kế và triển khai hệ thống.

5.1.2 Về mặt thực nghiệm

Về mặt thực nghiệm, đề tài đã xây dựng được một hệ thống ứng dụng có khả năng hỗ trợ sinh viên trong việc tìm kiếm địa điểm học tập, chia sẻ tài liệu và kết nối cộng đồng học tập trên cùng một nền tảng. Qua quá trình cài đặt và thử nghiệm, hệ thống cho thấy khả năng hoạt động tương đối ổn định ở các chức năng chính.

Cụ thể, ứng dụng đã thực hiện được các chức năng như đăng ký, đăng nhập, quản lý hồ sơ cá nhân, tìm kiếm và hiển thị địa điểm học tập trên bản đồ, xem và chia sẻ tài liệu, tạo nhóm học tập, trò chuyện nhóm, kết bạn, lưu danh sách yêu thích, gửi phản hồi và báo cáo nội dung. Các chức năng này cơ bản hoạt động đúng theo yêu cầu đề ra ban đầu và phản ánh được đúng định hướng xây dựng một nền tảng học tập cộng đồng dành cho sinh viên.

Về hiệu quả thực tế, hệ thống đã giải quyết được một số vấn đề mà bài toán đặt ra. Thứ nhất, ứng dụng giúp tập trung thông tin về địa điểm học tập và tài liệu học tập vào cùng một hệ thống, giúp người dùng thuận tiện hơn trong việc tra cứu và sử dụng. Thứ hai, hệ thống hỗ trợ kết nối cộng đồng thông qua nhóm học tập, kết bạn và trao đổi tài nguyên, góp phần tăng tính tương tác giữa các sinh viên. Thứ ba, việc kết hợp giữa backend, Firebase và SQLite giúp hệ thống có khả năng lưu trữ, đồng bộ và duy trì trạng thái sử dụng tương đối hiệu quả.

Kết quả thực nghiệm cũng cho thấy hệ thống có tính khả thi về mặt triển khai. Giao diện ứng dụng tương đối trực quan, dễ sử dụng, thời gian phản hồi ở các chức năng cơ bản ở mức chấp nhận được, và một số chức năng thời gian thực như chat nhóm có khả năng cập nhật nhanh. Điều này chứng tỏ đề tài không chỉ có giá trị về mặt học thuật mà còn có khả năng phát triển thêm để ứng dụng vào thực tế.

5.2 Hạn chế còn tồn đọng

Mặc dù đã đạt được một số kết quả tích cực, hệ thống vẫn còn tồn tại một số hạn chế nhất định.

Thứ nhất, hiệu năng của hệ thống vẫn phụ thuộc khá nhiều vào kết nối mạng, đặc biệt đối với các chức năng cần truy xuất dữ liệu từ backend, đồng bộ Firebase hoặc tải tệp. Trong điều kiện mạng không ổn định, tốc độ phản hồi có thể bị giảm, từ đó ảnh hưởng đến trải nghiệm sử dụng thực tế.

Thứ hai, do thời gian thực hiện đề tài có hạn, phạm vi kiểm thử của nhóm vẫn chưa thể bao quát toàn bộ các tình huống sử dụng, nhiều loại thiết bị hay số lượng người dùng lớn cùng lúc. Vì vậy, hệ thống vẫn có thể còn tồn tại một số lỗi tiềm ẩn chưa được phát hiện trong giai đoạn hiện tại.

Ngoài ra, việc tích hợp nhiều công nghệ khác nhau như Flutter, Firebase, Firestore, SQLite và backend API cũng làm tăng độ phức tạp của hệ thống. Điều này đòi hỏi quá trình bảo trì và mở rộng về sau cần được thực hiện cẩn thận để tránh phát sinh lỗi đồng bộ dữ liệu hoặc lỗi logic giữa các thành phần.

5.3 Hướng phát triển trong tương lai

Trong thời gian tới, hệ thống có thể tiếp tục được phát triển theo nhiều hướng để nâng cao chất lượng và tính ứng dụng thực tế.

Trước hết, nhóm có thể tiếp tục hoàn thiện các chức năng còn thiếu hoặc mới dừng ở mức cơ bản, chẳng hạn như quản lý phiên đăng nhập, xác thực hai lớp, lịch sử hoạt động, đánh giá ứng dụng hoặc các công cụ quản lý người dùng chi tiết hơn. Việc hoàn thiện các chức năng này sẽ giúp hệ thống trở nên đầy đủ và chuyên nghiệp hơn.

Thứ hai, cần tiếp tục tối ưu hiệu năng của hệ thống, đặc biệt ở các chức năng tải dữ liệu, đồng bộ thời gian thực và xử lý tệp. Ngoài ra, có thể bổ sung cơ chế cache tốt hơn, tối ưu truy vấn dữ liệu và cải thiện trải nghiệm sử dụng trong điều kiện mạng yếu.

Thứ ba, hệ thống có thể được mở rộng thêm các tính năng thông minh hơn như gợi ý địa điểm học tập phù hợp, đề xuất tài liệu liên quan, gợi ý nhóm học tập theo sở thích hoặc học phần, cũng như cá nhân hóa nội dung hiển thị cho từng người dùng. Đây là hướng phát triển phù hợp để nâng cao tính hữu ích và khả năng hỗ trợ học tập của ứng dụng.

Bên cạnh đó, nhóm cũng có thể phát triển thêm các công cụ đánh giá và thống kê, ví dụ như biểu đồ hoạt động học tập, mức độ tương tác trong nhóm, số lượng tài liệu chia sẻ hoặc hiệu quả sử dụng địa điểm học tập. Những chức năng này sẽ giúp hệ thống không chỉ là công cụ hỗ trợ sử dụng hằng ngày mà còn hỗ trợ phân tích và cải thiện hoạt động học tập cộng đồng.

Cuối cùng, nếu có điều kiện triển khai thực tế, hệ thống cần được kiểm thử trên quy mô lớn hơn, với nhiều nhóm người dùng và nhiều thiết bị khác nhau để đánh giá độ ổn định, bảo mật và khả năng mở rộng. Đây sẽ là bước quan trọng để đưa hệ thống từ mức đồ án học thuật tiến gần hơn tới một sản phẩm thực tế.

Kết luận chương: Chương này đã tổng kết những kết quả chính mà đề tài đạt được cả về mặt lý thuyết lẫn thực nghiệm, đồng thời chỉ ra các hạn chế còn tồn tại trong quá trình triển khai hệ thống. Trên cơ sở đó, chương cũng đề xuất một số hướng phát triển trong tương lai nhằm tiếp tục hoàn thiện chức năng, tối ưu hiệu năng và nâng cao tính ứng dụng thực tế của hệ thống.

TÀI LIỆU THAM KHẢO

- [1]. [Google LLC, “Flutter Documentation”, flutter.dev, 2024](#)
- [2]. [Meta Platforms, “React – A JavaScript library for building user interfaces”, react.dev, 2024](#)
- [3]. [OpenJSFoundation, “Node.js Documentation”, nodejs.org, 2024](#)
- [4]. [Google LLC, “Firebase Documentation”, firebase.google.com, 2024](#)